

capstone_project_2_final

June 1, 2026

1 GreenCity Logistics Prescriptive Analytics Capstone

1.1 Client Case Study 1: E-Commerce Provider C103

1.1.1 TEAM: Natalie Avina, Navya Bhatia, Joane Sarfati, Mrudhika Sirineni, and Mya Stewart

1.2 1. Table of Contents

- Section 2 — Packages
- Section 3 — Load Data
- Section 4 — Data Cleaning
- Section 5 — EDA
- Section 6 — Zone Route Matrix
- Section 7 — LP Model
- Section 8 — MIP Formulation & Client Case Study C103
- Section 9 — Network Flow Model
- Section 10 — Goal Programming Model
- Section 11 — Pareto Frontier Analysis
- Section 12 — Sensitivity / Scenario Analysis
- Section 13 — Managerial Insights
- Section 14 — Limitations & Improvements
- Section 15 — User Guide

1.2.1 C103 Requirements Modeled

At most 6 stops per trip

We model one planned daily route per open facility. Each open facility can serve at most 6 demand zones. This is a conservative approximation of the C103 6-stop route limit.

Electric vehicles in certain districts

Any zone designated as a low-emission zone can only be assigned an electric vehicle.

Delivery within 2-hour windows

Facility–zone–vehicle assignments are allowed only if travel time is 120 minutes or less and the selected vehicle has sufficient range.

Either serve a zone completely or not at all

The C103 model serves each selected zone completely. Each zone is assigned to exactly one facility and one vehicle type.

```
[3]: # 2. Install Packages
!pip install gurobipy
!pip install gurobipy --quiet
!pip install folium --quiet
!pip install plotly --quiet

print('All packages installed.')
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import matplotlib.colors as mcolors
from matplotlib.gridspec import GridSpec
import seaborn as sns
import networkx as nx
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
import warnings
import math
warnings.filterwarnings('ignore')

from sklearn.cluster import KMeans
from sklearn.preprocessing import MinMaxScaler

from gurobipy import Model, GRB, quicksum
import os
os.chdir("/Users/mya/Downloads/scu/msba/
↳isba2405-prescriptive-analytics-w-python")
#Global Plot Style
plt.rcParams.update({
    'figure.facecolor': '#0f1117',
    'axes.facecolor': '#1a1d2e',
    'axes.edgecolor': '#2e3250',
    'axes.labelcolor': '#c8d0e7',
    'xtick.color': '#8892b0',
    'ytick.color': '#8892b0',
    'text.color': '#c8d0e7',
    'grid.color': '#2e3250',
    'grid.linestyle': '--',
    'grid.alpha': 0.5,
    'font.family': 'DejaVu Sans',
    'axes.titlesize': 14,
    'axes.labelsize': 11,
})

PALETTE = {
```

```

    'teal':    '#64ffda',
    'blue':    '#57a5f8',
    'pink':    '#ff79c6',
    'orange':  '#ffb86c',
    'purple':  '#bd93f9',
    'green':   '#50fa7b',
    'red':     '#ff5555',
    'yellow':  '#f1fa8c',
}
COLORS = list(PALETTE.values())
RANDOM_SEED = 42
np.random.seed(RANDOM_SEED)
print('Libraries imported and style configured.')

```

Requirement already satisfied: gurobipy in /opt/anaconda3/lib/python3.12/site-packages (13.0.2)

All packages installed.

Libraries imported and style configured.

This section prepares the technical environment needed to build the full optimization decision system. The imported libraries support different parts of the project: pandas and NumPy are used for data preparation, matplotlib, seaborn, and Plotly support visual analysis, NetworkX supports network visualization, scikit-learn supports demand clustering, and Gurobi is used to formulate and solve the optimization models. Setting the plotting style at the beginning also makes the visual outputs more consistent and professional throughout the notebook.

From a business perspective, this section establishes the analytical foundation for the project. Since the project requires data analysis, mathematical optimization, visualization, and decision support, the package setup ensures that the team can move from raw operational data to interpretable business recommendations in one reproducible workflow.

1.3 3. Load the project data

Our project uses five datasets

- location_data.csv
- demand_data.csv
- vehicle_data.csv
- distance_matrix_SAMPLE_50routes.csv
- emission_constraints.csv

```

[6]: #Load Data
location_df = pd.read_csv('location_data.csv')
demand_df   = pd.read_csv('demand_data.csv')
vehicle_df  = pd.read_csv('vehicle_data.csv')
distance_df = pd.read_csv('distance_matrix_SAMPLE_50routes.csv') #updated_
↪filename
emission_df = pd.read_csv('emission_constraints.csv')

```

```

print('Locations:', location_df.shape)
print('Demand Zones:', demand_df.shape)
print('Vehicles:', vehicle_df.shape)
print('Distances:', distance_df.shape, '(50-route sample - L001-L005 x Z001-Z010)')
print('Emissions:', emission_df.shape)

print("Data loaded.")

```

```

Locations: (25, 12)
Demand Zones: (30, 11)
Vehicles: (10, 12)
Distances: (50, 8) (50-route sample - L001-L005 x Z001-Z010)
Emissions: (30, 10)
Data loaded.

```

This section loads the five main datasets used in the GreenCity Logistics optimization problem: facility locations, customer demand zones, vehicle options, route distances, and emission constraints. Together, these datasets represent the major components of the last-mile logistics system. Facility data describes where fulfillment centers could be opened, demand data describes where customer packages must be delivered, vehicle data describes fleet options, the distance matrix supports route-level cost and travel-time calculations, and the emission constraints identify zones with environmental restrictions.

The loaded data also shows an important limitation: the original distance file contains only 50 sample routes, while the full network requires all 25 facilities and 30 zones. This is addressed later by generating a complete 750-route matrix. Identifying this limitation early is important because optimization results depend heavily on having a complete and consistent route network.

```

[8]: # 4. Data Cleaning
for df in [location_df, demand_df, vehicle_df, distance_df, emission_df]:
    df.drop_duplicates(inplace=True)
    df.fillna(0, inplace=True)

demand_df.columns = demand_df.columns.str.strip().str.lstrip('\ufeff')
emission_df.columns = emission_df.columns.str.strip()

if emission_df['low_emission_zone'].dtype == object:
    emission_df['low_emission_zone'] = (emission_df['low_emission_zone'] == 'Yes').astype(int)

print('Data cleaned.')
display(location_df.head(3))
display(demand_df.head(3))

```

Data cleaned.

	location_id	latitude	longitude	district	building_type	\
0	L001	40.7128	-73.9960	Downtown	Warehouse	
1	L002	40.7148	-73.9980	Downtown	Retail Space	

2	L003	40.7165	-73.9945	Downtown	Mixed-Use
---	------	---------	----------	----------	-----------

	available_area_sqm	max_capacity_packages	setup_cost	monthly_lease	\
0	850	5000	480000	12500	
1	420	2500	320000	18000	
2	650	3800	410000	15000	

	utility_cost	suitability_score	zoning_restriction
0	3500	0.92	Low
1	2800	0.85	Moderate
2	3200	0.88	Moderate

	zone_id	latitude	longitude	district	population_density	\
0	Z001	40.7129	-73.9958	Downtown	21500	
1	Z002	40.7145	-73.9980	Downtown	20800	
2	Z003	40.7168	-73.9945	Downtown	19500	

	daily_demand_packages	priority_delivery_percent	time_window_start	\
0	350	42	8	
1	325	38	9	
2	310	45	8	

	time_window_end	delivery_difficulty	weekend_demand_factor
0	20	0.65	1.2
1	21	0.70	1.3
2	22	0.68	1.4

This section improves the reliability and consistency of the input data before optimization. Duplicate records are removed, missing values are handled, and column names are standardized so the same variables can be used consistently across models. The low-emission zone field is also converted into a numeric indicator, which allows it to be used directly in optimization constraints.

These cleaning steps are important because even small inconsistencies in the data can create infeasible models, incorrect assignments, or misleading results. From a business standpoint, the cleaning process reduces the risk of making facility or vehicle decisions based on incomplete or inconsistent information. This section supports the technical correctness requirement of the project by ensuring that the optimization models are built on dependable inputs.

1.4 5. Exploratory Data Analysis

We conducted EDA to better understand demand concentration, facility cost-capacity trade-offs, vehicle emissions, and geographic coverage before building optimization models.

```
[11]: # 5. Exploratory data analysis

fig, axes = plt.subplots(2, 2, figsize=(16, 11))
fig.patch.set_facecolor('#0f1117')
fig.suptitle('GreenCity Logistics - Data Overview Dashboard',
             fontsize=17, color=PALETTE['teal'], fontweight='bold', y=1.01)
```

```

# Plot 1: Demand distribution
ax1 = axes[0, 0]
ax1.set_facecolor('#1a1d2e')
sns.histplot(demand_df['daily_demand_packages'], bins=12,
              color=PALETTE['teal'], alpha=0.85, ax=ax1, edgecolor='#0f1117')
ax1.axvline(demand_df['daily_demand_packages'].mean(), color=PALETTE['pink'],
            linestyle='--', linewidth=1.8,
            label=f"Mean = {demand_df['daily_demand_packages'].mean():.0f}")
ax1.set_title('Daily Demand Distribution (Packages)', color='#c8d0e7', pad=10)
ax1.set_xlabel('Packages / Day')
ax1.set_ylabel('Frequency')
ax1.legend(facecolor='#1a1d2e', edgecolor='#2e3250', labelcolor='#c8d0e7')
ax1.grid(True)

# Plot 2: Facility setup cost vs capacity
ax2 = axes[0, 1]
ax2.set_facecolor('#1a1d2e')
sc = ax2.scatter(location_df['max_capacity_packages'],
                  location_df['setup_cost'] / 1000,
                  c=location_df.get('suitability_score', pd.Series([0.5] *
↪ len(location_df))),
                  cmap='plasma', s=90, alpha=0.9, edgecolors='#2e3250',
↪ linewidths=0.6)
cb = fig.colorbar(sc, ax=ax2, fraction=0.035)
cb.set_label('Suitability Score', color='#c8d0e7')
cb.ax.yaxis.set_tick_params(color='#8892b0')
plt.setp(plt.getp(cb.ax.axes, 'yticklabels'), color='#8892b0')

for _, row in location_df.iterrows():
    ax2.annotate(row['location_id'],
                  (row['max_capacity_packages'], row['setup_cost'] / 1000),
                  fontsize=5, color='#8892b0', ha='center', va='bottom')
ax2.set_title('All Facilities: Capacity vs Setup Cost', color='#c8d0e7', pad=10)
ax2.set_xlabel('Max Capacity (packages)')
ax2.set_ylabel('Setup Cost ($000s)')
ax2.grid(True)

# Plot 3: Vehicle CO2 emissions
ax3 = axes[1, 0]
ax3.set_facecolor('#1a1d2e')
bar_colors = [PALETTE['green'] if str(ft).lower() == 'electric' or 'Electric'
↪ in str(v)
               else PALETTE['orange'] if str(ft).lower() == 'hybrid' or 'Hybrid'
↪ in str(v)
               else PALETTE['red']]

```

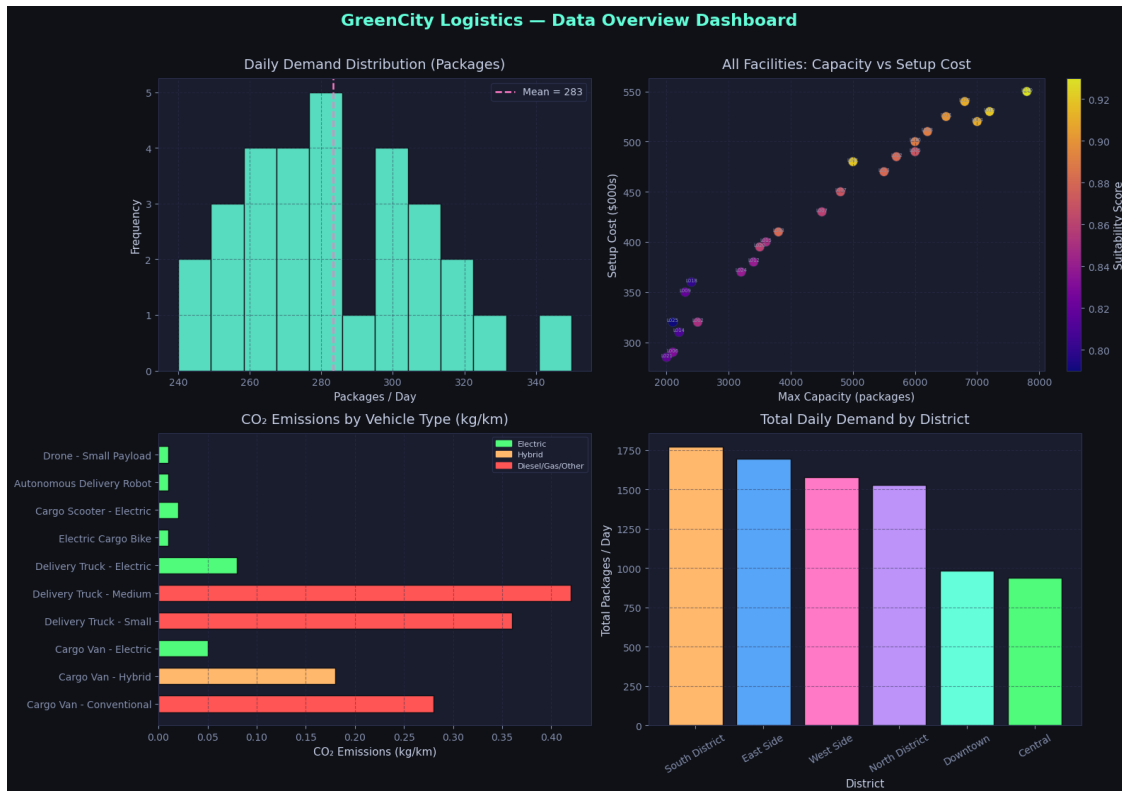
```

        for v, ft in zip(vehicle_df['vehicle_type'], vehicle_df.
        ↪get('fuel_type', [''] * len(vehicle_df)))
ax3.barh(vehicle_df['vehicle_type'],
        vehicle_df['co2_emissions_kg_per_km'],
        color=bar_colors, edgecolor='#0f1117', height=0.6)
ax3.set_title('CO Emissions by Vehicle Type (kg/km)', color='#c8d0e7', pad=10)
ax3.set_xlabel('CO Emissions (kg/km)')
legend_patches = [
    mpatches.Patch(color=PALETTE['green'], label='Electric'),
    mpatches.Patch(color=PALETTE['orange'], label='Hybrid'),
    mpatches.Patch(color=PALETTE['red'], label='Diesel/Gas/Other'),
]
ax3.legend(handles=legend_patches, facecolor='#1a1d2e',
        edgecolor='#2e3250', labelcolor='#c8d0e7', fontsize=8)
ax3.grid(True, axis='x')

# Plot 4: Demand by district
ax4 = axes[1, 1]
ax4.set_facecolor('#1a1d2e')
district_colors_map = {
    'Downtown': '#64ffda', 'East Side': '#57a5f8', 'West Side': '#ff79c6',
    'South District': '#ffb86c', 'North District': '#bd93f9', 'Central': '
    ↪'#50fa7b'
}
dist_demand = demand_df.groupby('district')['daily_demand_packages'].sum().
    ↪sort_values(ascending=False)
dist_cols = [district_colors_map.get(d, '#ffffff') for d in dist_demand.index]
ax4.bar(dist_demand.index, dist_demand.values, color=dist_cols,
    ↪edgecolor='#0f1117')
ax4.set_title('Total Daily Demand by District', color='#c8d0e7', pad=10)
ax4.set_xlabel('District')
ax4.set_ylabel('Total Packages / Day')
ax4.tick_params(axis='x', rotation=30)
ax4.grid(True, axis='y')

plt.tight_layout()
plt.show()

```



The exploratory data analysis provides a first look at the operational structure of the GreenCity network. The demand distribution shows how daily package volume varies across zones, while the district-level demand chart identifies which geographic areas create the greatest delivery burden. The facility cost-capacity chart compares potential fulfillment centers based on setup cost, maximum capacity, and suitability. The vehicle emissions chart highlights the environmental differences between electric, hybrid, and traditional vehicle options.

[13]: # 5.1 Geographic map and clustering charts

```
# Interactive map: all facilities and all demand zones
fig_map = go.Figure()
district_colors = {
    'Downtown': '#64ffda', 'East Side': '#57a5f8',
    'West Side': '#ff79c6', 'South District': '#ffb86c',
    'North District': '#bd93f9', 'Central': '#50fa7b'
}

for dist in demand_df['district'].unique():
    sub = demand_df[demand_df['district'] == dist]
    fig_map.add_trace(go.Scattermapbox(
        lat=sub['latitude'], lon=sub['longitude'], mode='markers',
        marker=dict(size=sub['daily_demand_packages'] / 18,
```



```

        color=district_colors.get(dist, '#ffffff'), opacity=0.75),
        text=sub['zone_id'] + '<br>District: ' + sub['district'] +
            '<br>Demand: ' + sub['daily_demand_packages'].astype(str) + ' pkgs/
↪day',
        hoverinfo='text', name=f'Zone - {dist}',
    ))

fig_map.add_trace(go.Scattermapbox(
    lat=location_df['latitude'], lon=location_df['longitude'],
    ↪mode='markers+text',
    marker=dict(size=13, color=PALETTE['pink'], symbol='square'),
    text=location_df['location_id'], textposition='top right',
    textfont=dict(size=8, color='white'),
    hovertext=(location_df['location_id'] + '<br>' + location_df['district'] +
        '<br>Capacity: ' + location_df['max_capacity_packages'].
    ↪astype(str) +
        '<br>Setup: $' + location_df['setup_cost'].astype(str)),
    hoverinfo='text', name=f'Potential Facility (all {len(location_df)})',
))

if 'open_c103' in globals():
    open_locs = location_df[location_df['location_id'].isin(open_c103)]
    fig_map.add_trace(go.Scattermapbox(
        lat=open_locs['latitude'], lon=open_locs['longitude'],
    ↪mode='markers+text',
        marker=dict(size=18, color=PALETTE['green'], symbol='star'),
        text=open_locs['location_id'], textposition='bottom right',
        textfont=dict(size=10, color='white'),
        hovertext='C103 selected facility: ' + open_locs['location_id'],
        hoverinfo='text', name='C103 Selected Facilities',
    ))

fig_map.update_layout(
    mapbox=dict(style='carto-darkmatter',
        center=dict(lat=location_df['latitude'].mean(),
    ↪lon=location_df['longitude'].mean()),
        zoom=12),
    paper_bgcolor='#0f1117', font_color='#c8d0e7',
    title=dict(text=f'GreenCity - All {len(location_df)} Facilities &
    ↪{len(demand_df)} Demand Zones',
        font=dict(size=15, color=PALETTE['teal'])),
    legend=dict(bgcolor='#1a1d2e', bordercolor='#2e3250', font=dict(size=9)),
    margin=dict(l=0, r=0, t=50, b=0), height=580,
)
fig_map.show()

```

```

# Clustering charts
cluster_features = demand_df[['population_density', 'daily_demand_packages']]
scaler = MinMaxScaler()
scaled = scaler.fit_transform(cluster_features)

inertias = []
K_range = range(2, 9)
for k in K_range:
    km = KMeans(n_clusters=k, random_state=RANDOM_SEED, n_init=10)
    km.fit(scaled)
    inertias.append(km.inertia_)

kmeans = KMeans(n_clusters=4, random_state=RANDOM_SEED, n_init=10)
demand_df['cluster'] = kmeans.fit_predict(scaled)
cluster_labels = {
    0: 'High Density/High Demand', 1: 'High Density/Low Demand',
    2: 'Low Density/High Demand', 3: 'Low Density/Low Demand'
}
demand_df['cluster_label'] = demand_df['cluster'].map(cluster_labels)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))
fig.patch.set_facecolor('#0f1117')
ax1.set_facecolor('#1a1d2e')
ax1.plot(list(K_range), inertias, color=PALETTE['teal'], marker='o',
         markersize=8, linewidth=2, markerfacecolor=PALETTE['pink'])
ax1.axvline(4, color=PALETTE['yellow'], linestyle='--', linewidth=1.5,
           label='Chosen K=4')
ax1.set_title('Elbow Method - Optimal K Selection', color='#c8d0e7', pad=10)
ax1.set_xlabel('Number of Clusters (K)')
ax1.set_ylabel('Inertia')
ax1.legend(facecolor='#1a1d2e', edgecolor='#2e3250', labelcolor='#c8d0e7')
ax1.grid(True)

ax2.set_facecolor('#1a1d2e')
cluster_colors = [PALETTE['teal'], PALETTE['pink'], PALETTE['orange'],
                  PALETTE['purple']]
for c, col in zip(sorted(demand_df['cluster'].unique()), cluster_colors):
    mask = demand_df['cluster'] == c
    ax2.scatter(demand_df.loc[mask, 'population_density'],
               demand_df.loc[mask, 'daily_demand_packages'],
               color=col, s=80, label=cluster_labels.get(c, f'Cluster {c}'),
               alpha=0.9, edgecolors='#0f1117')
ax2.set_title('Demand Zone Clusters', color='#c8d0e7', pad=10)
ax2.set_xlabel('Population Density')
ax2.set_ylabel('Daily Demand (packages)')
ax2.legend(facecolor='#1a1d2e', edgecolor='#2e3250', labelcolor='#c8d0e7',
           fontsize=8)

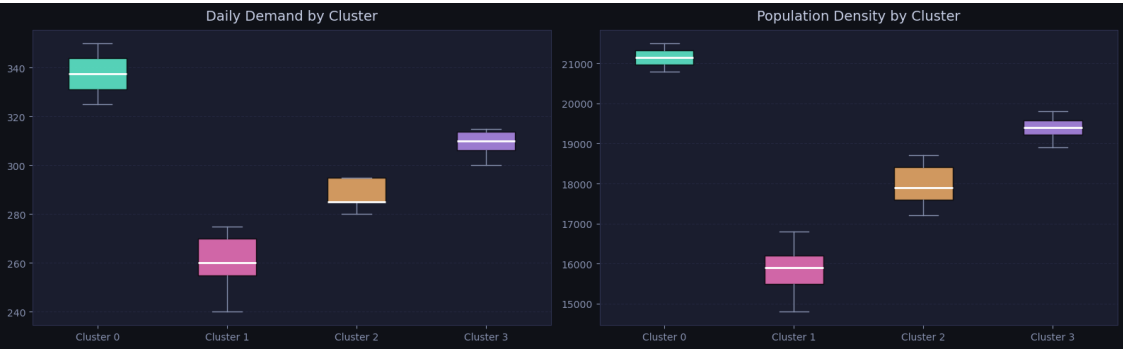
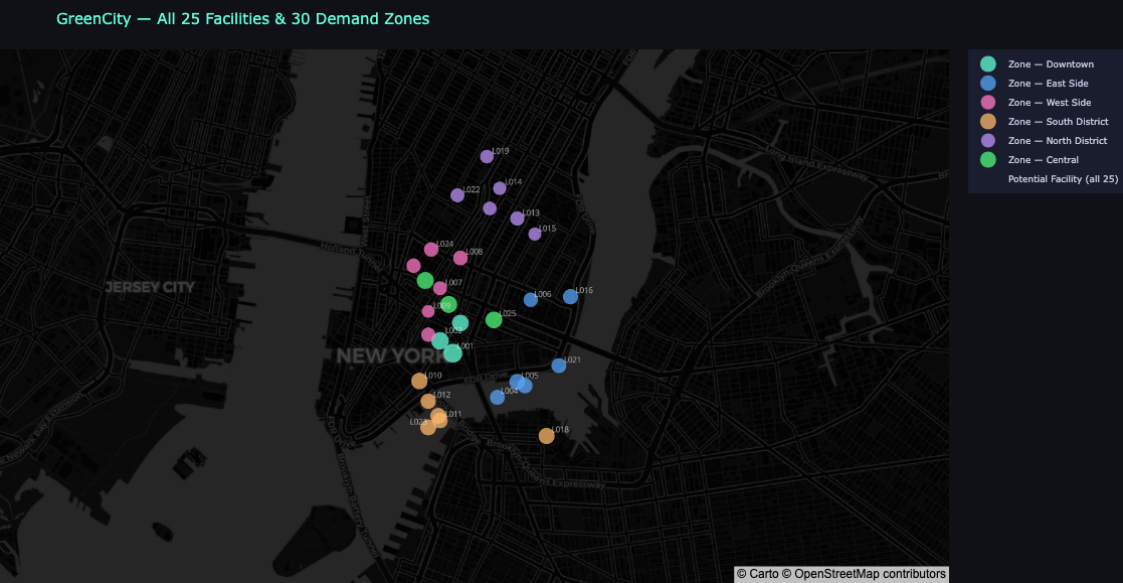
```

```

ax2.grid(True)
plt.tight_layout()
plt.savefig('clustering_original_style.png', dpi=150, bbox_inches='tight',
    ↳facecolor='#0f1117')
plt.show()

# Cluster boxplots
fig, axes = plt.subplots(1, 2, figsize=(16, 5))
fig.patch.set_facecolor('#0f1117')
for ax, col, title in zip(
    axes,
    ['daily_demand_packages', 'population_density'],
    ['Daily Demand by Cluster', 'Population Density by Cluster']
):
    ax.set_facecolor('#1a1d2e')
    sorted_clusters = sorted(demand_df['cluster'].unique())
    bp = ax.boxplot(
        [demand_df.loc[demand_df['cluster'] == c, col].values for c in
    ↳sorted_clusters],
        patch_artist=True,
        medianprops=dict(color='white', linewidth=2),
        whiskerprops=dict(color='#8892b0'),
        capprops=dict(color='#8892b0'),
        flierprops=dict(marker='o', color=PALETTE['red'], markersize=4),
    )
    for patch, color in zip(bp['boxes'], cluster_colors):
        patch.set_facecolor(color)
        patch.set_alpha(0.8)
    ax.set_xticklabels([f'Cluster {c}' for c in sorted_clusters])
    ax.set_title(title, color='#c8d0e7', pad=10)
    ax.grid(True, axis='y')
plt.tight_layout()
plt.savefig('cluster_boxplots_original_style.png', dpi=150,
    ↳bbox_inches='tight', facecolor='#0f1117')
plt.show()
print('Map and clustering visuals rendered.')

```



Map and clustering visuals rendered.

The geographic map and clustering analysis add spatial context to the decision problem. Clustering demand zones by population density and package demand helps identify areas that may require stronger facility coverage or more frequent delivery service. This is important because logistics decisions are not based only on total demand; they also depend on where demand is located relative to potential facilities.

Overall, this section helps justify why optimization is needed. The network contains multiple conflicting factors: facilities vary in cost and capacity, zones vary in demand, vehicles vary in cost and emissions, and environmental restrictions vary by location. The EDA confirms that GreenCity needs a structured optimization approach rather than a simple manual facility selection.

1.5 6. Create a full facility-zone route matrix

```
[16]: # 6. Full route matrix and parameter data
# This section creates a complete facility-to-zone route matrix.
#
# If the uploaded distance matrix is incomplete, we generate all 25 x 30 routes.
# Travel time is estimated using the nonlinear travel-time function from the
# ↪project guide:
#  $t(x) = t_0 * (1 + \alpha * (x / C)^\beta)$ 
#
# where:
#  $t_0$  = free-flow travel time
#  $x$  = demand flow proxy for the zone
#  $C$  = capacity proxy for the facility
#  $\alpha, \beta$  = congestion sensitivity parameters

def haversine_km(lat1, lon1, lat2, lon2):
    """Calculate great-circle distance in kilometers."""
    R = 6371.0
    lat1, lon1, lat2, lon2 = map(math.radians, [lat1, lon1, lat2, lon2])
    dlat = lat2 - lat1
    dlon = lon2 - lon1

    a = (
        math.sin(dlat / 2) ** 2
        + math.cos(lat1) * math.cos(lat2) * math.sin(dlon / 2) ** 2
    )

    return 2 * R * math.atan2(math.sqrt(a), math.sqrt(1 - a))

# Required distance columns

required_distance_cols = {"origin", "destination", "distance_km"}

if not required_distance_cols.issubset(distance_df.columns):
```

```

    raise ValueError(f"Distance matrix must contain these columns:␣
↳{required_distance_cols}")

# Route matrix dimensions

n_expected = (
    location_df["location_id"].nunique()
    * demand_df["zone_id"].nunique()
)

n_observed = (
    distance_df[["origin", "destination"]]
    .drop_duplicates()
    .shape[0]
)

# Nonlinear travel-time settings

BASE_SPEED_KMPH = 25
ALPHA = 0.15
BETA = 4

def nonlinear_travel_time_mins(distance_km, flow, capacity):
    """
    Estimate travel time using the nonlinear congestion function:

        
$$t(x) = t_0 * (1 + \alpha * (x / C)^\beta)$$


    distance_km = route distance
    flow        = demand flow proxy
    capacity    = facility capacity proxy
    """

    free_flow_time = (distance_km / BASE_SPEED_KMPH) * 60
    capacity = max(capacity, 1)

    congestion_factor = 1 + ALPHA * (flow / capacity) ** BETA
    travel_time = free_flow_time * congestion_factor

    return travel_time, congestion_factor

```

```

# Case 1: uploaded distance matrix is already complete

if n_observed >= n_expected:
    distance_df1 = distance_df.copy()

    print(f"Using uploaded complete distance matrix with {n_observed} routes.")

    # If travel_time_mins is missing, calculate it using the nonlinear function.
    if "travel_time_mins" not in distance_df1.columns:
        print("travel_time_mins missing. Calculating nonlinear travel times.")

        distance_df1 = distance_df1.merge(
            demand_df[["zone_id", "daily_demand_packages"]],
            left_on="destination",
            right_on="zone_id",
            how="left"
        )

        distance_df1 = distance_df1.merge(
            location_df[["location_id", "max_capacity_packages"]],
            left_on="origin",
            right_on="location_id",
            how="left"
        )

        travel_results = distance_df1.apply(
            lambda r: nonlinear_travel_time_mins(
                r["distance_km"],
                r["daily_demand_packages"],
                r["max_capacity_packages"]
            ),
            axis=1
        )

        distance_df1["travel_time_mins"] = [
            round(result[0], 1) for result in travel_results
        ]

        distance_df1["congestion_factor"] = [
            round(result[1], 3) for result in travel_results
        ]

        distance_df1.drop(
            columns=["zone_id", "location_id", "daily_demand_packages",
            ↪ "max_capacity_packages"],
            inplace=True,

```

```

        errors="ignore"
    )

    # If congestion factor is missing, estimate it from travel time and
    ↪ free-flow time.
    if "congestion_factor" not in distance_df1.columns:
        distance_df1["free_flow_time_mins"] = (distance_df1["distance_km"] /
        ↪ BASE_SPEED_KMPH) * 60
        distance_df1["congestion_factor"] = (
            distance_df1["travel_time_mins"] /
            ↪ distance_df1["free_flow_time_mins"].replace(0, np.nan)
        ).fillna(1.0)
        distance_df1["congestion_factor"] = distance_df1["congestion_factor"].
        ↪ round(3)
        distance_df1.drop(columns=["free_flow_time_mins"], inplace=True)

    if "emission_factor" not in distance_df1.columns:
        distance_df1["emission_factor"] = (
            1.0 + 0.5 * (distance_df1["congestion_factor"] - 1.0)
        ).round(3)

    if "toll_cost" not in distance_df1.columns:
        distance_df1["toll_cost"] = 0.0

    if "time_reliability" not in distance_df1.columns:
        distance_df1["time_reliability"] = (
            1.0 - 0.01 * distance_df1["distance_km"]
        ).clip(lower=0.75).round(3)

# Case 2: uploaded distance matrix is incomplete

else:
    print(f"Uploaded distance file has {n_observed} routes, but {n_expected}
    ↪ are needed.")
    print("Generating complete route matrix from coordinates using nonlinear
    ↪ travel time.")

    rows = []

    for _, loc in location_df.iterrows():
        for _, zon in demand_df.iterrows():

            dist = haversine_km(

```



```

        loc["latitude"],
        loc["longitude"],
        zon["latitude"],
        zon["longitude"]
    )

    flow = zon["daily_demand_packages"]
    capacity = loc["max_capacity_packages"]

    travel, congestion_factor = nonlinear_travel_time_mins(
        distance_km=dist,
        flow=flow,
        capacity=capacity
    )

    rows.append({
        "origin": loc["location_id"],
        "destination": zon["zone_id"],
        "distance_km": round(dist, 3),
        "travel_time_mins": round(travel, 1),
        "congestion_factor": round(congestion_factor, 3),
        "emission_factor": round(1.0 + 0.5 * (congestion_factor - 1.0), 3),
        "toll_cost": 0.0,
        "time_reliability": max(0.75, round(1.0 - 0.01 * dist, 3)),
    })

    distance_df1 = pd.DataFrame(rows)

# Active model sets

facilities = sorted(location_df["location_id"].unique().tolist())
zones = sorted(demand_df["zone_id"].unique().tolist())
vehicles = sorted(vehicle_df["vehicle_type"].unique().tolist())

# Final checks and output

print(f"Active facilities: {len(facilities)}")
print(f"Active zones: {len(zones)}")
print(f"Active vehicle types: {len(vehicles)}")
print(f"Routes: {len(distance_df1)}")

```

```

print()
print("Nonlinear travel-time function used:")
print("t(x) = t0 * (1 + alpha * (x / C)^beta)")
print(f"Base speed: {BASE_SPEED_KMPH} km/h")
print(f"Alpha: {ALPHA}")
print(f"Beta: {BETA}")

display(distance_df1.head())

```

Uploaded distance file has 50 routes, but 750 are needed.

Generating complete route matrix from coordinates using nonlinear travel time.

Active facilities: 25

Active zones: 30

Active vehicle types: 10

Routes: 750

Nonlinear travel-time function used:

$t(x) = t_0 * (1 + \alpha * (x / C)^\beta)$

Base speed: 25 km/h

Alpha: 0.15

Beta: 4

	origin	destination	distance_km	travel_time_mins	congestion_factor \
0	L001	Z001	0.020	0.0	1.0
1	L001	Z002	0.253	0.6	1.0
2	L001	Z003	0.462	1.1	1.0
3	L001	Z004	0.906	2.2	1.0
4	L001	Z005	1.025	2.5	1.0

	emission_factor	toll_cost	time_reliability
0	1.0	0.0	1.000
1	1.0	0.0	0.997
2	1.0	0.0	0.995
3	1.0	0.0	0.991
4	1.0	0.0	0.990

This section creates the complete route network used by the optimization models. Since the uploaded distance file contains only 50 routes, the notebook generates the full 25 facility by 30 demand zone matrix, resulting in 750 possible facility-zone connections. Distances are estimated using the haversine formula, and travel time is adjusted using a congestion factor based on population density.

This step is essential because the optimization model must be able to evaluate every possible facility-to-zone assignment. Without the full route matrix, some zones or facilities would be excluded from the model simply because their route data was missing. That would make the recommendation less reliable.

[18]: *# 6.1 Optimization parameters*

```

# Facility parameters
facility_setup_cost = dict(zip(location_df["location_id"],
    ↪location_df["setup_cost"]))
facility_capacity = dict(zip(location_df["location_id"],
    ↪location_df["max_capacity_packages"]))
facility_monthly_lease = dict(zip(location_df["location_id"], location_df.
    ↪get("monthly_lease", pd.Series([0] * len(location_df)))))
facility_utility = dict(zip(location_df["location_id"], location_df.
    ↪get("utility_cost", pd.Series([0] * len(location_df)))))
facility_suitability = dict(zip(location_df["location_id"], location_df.
    ↪get("suitability_score", pd.Series([1] * len(location_df)))))

# Demand parameters
zone_demand = dict(zip(demand_df["zone_id"],
    ↪demand_df["daily_demand_packages"]))
zone_district = dict(zip(demand_df["zone_id"], demand_df.get("district", pd.
    ↪Series(["Unknown"] * len(demand_df)))))
time_window_start = dict(zip(demand_df["zone_id"], demand_df.
    ↪get("time_window_start", pd.Series([8] * len(demand_df)))))
time_window_end = dict(zip(demand_df["zone_id"], demand_df.
    ↪get("time_window_end", pd.Series([20] * len(demand_df)))))

# Vehicle parameters
vehicle_capacity = dict(zip(vehicle_df["vehicle_type"],
    ↪vehicle_df["capacity_packages"]))
vehicle_emission = dict(zip(vehicle_df["vehicle_type"],
    ↪vehicle_df["co2_emissions_kg_per_km"]))
vehicle_op_cost = dict(zip(vehicle_df["vehicle_type"],
    ↪vehicle_df["operating_cost_per_km"]))
vehicle_range = dict(zip(vehicle_df["vehicle_type"], vehicle_df.
    ↪get("max_range_km", pd.Series([10**6] * len(vehicle_df)))))
vehicle_purchase = dict(zip(vehicle_df["vehicle_type"], vehicle_df.
    ↪get("purchase_cost", pd.Series([0] * len(vehicle_df)))))
vehicle_life = dict(zip(vehicle_df["vehicle_type"], vehicle_df.
    ↪get("lifespan_years", pd.Series([5] * len(vehicle_df)).replace(0, 5))))
vehicle_maintenance = dict(zip(vehicle_df["vehicle_type"], vehicle_df.
    ↪get("maintenance_cost_annual", pd.Series([0] * len(vehicle_df)))))
vehicle_fuel = dict(zip(vehicle_df["vehicle_type"], vehicle_df.get("fuel_type",
    ↪pd.Series(["Unknown"] * len(vehicle_df)))))

# Emission parameters
zone_emission_cap = dict(zip(emission_df["zone_id"], emission_df.
    ↪get("emission_cap_kg_per_day", pd.Series([10**9] * len(emission_df)))))
low_emission_zones = emission_df.loc[emission_df.get("low_emission_zone", 0) ==
    ↪1, "zone_id"].tolist()

```

```

low_emission_zones = [z for z in low_emission_zones if z in zones]
electric_vehicles = [v for v in vehicles if str(vehicle_fuel[v]).strip().
    ↪lower() == "electric" or "electric" in v.lower()]
non_electric_vehicles = [v for v in vehicles if v not in electric_vehicles]

# Route parameters
route_distance = {(r.origin, r.destination): float(r.distance_km) for r in
    ↪distance_df1.itertuples()}
route_time = {(r.origin, r.destination): float(r.travel_time_mins) for r in
    ↪distance_df1.itertuples()}
route_emission_factor = {(r.origin, r.destination): float(getattr(r,
    ↪"emission_factor", 1.0)) for r in distance_df1.itertuples()}
route_reliability = {(r.origin, r.destination): float(getattr(r,
    ↪"time_reliability", 0.95)) for r in distance_df1.itertuples()}

# Dailyized facility and vehicle fixed costs.
# Setup cost is annualized over 5 years, monthly lease and utility are
    ↪converted to daily costs.
facility_daily_fixed_cost = {
    i: facility_setup_cost[i] / (5 * 365) + (facility_monthly_lease[i] +
    ↪facility_utility[i]) / 30
    for i in facilities
}
vehicle_daily_fixed_cost = {
    v: vehicle_purchase[v] / max(vehicle_life[v] * 365, 1) +
    ↪vehicle_maintenance[v] / 365
    for v in vehicles
}

# Conservative assumptions
MAX_STOPS_PER_ROUTE = 6
MAX_DELIVERY_WINDOW_MINS = 120
MIN_TIME_RELIABILITY = 0.75

print(f"Low-emission zones covered: {len(low_emission_zones)}")
print(f"Electric vehicle types: {electric_vehicles}")
print("Parameter setup complete.")

```

Low-emission zones covered: 24

Electric vehicle types: ['Autonomous Delivery Robot', 'Cargo Scooter - Electric', 'Cargo Van - Electric', 'Delivery Truck - Electric', 'Drone - Small Payload', 'Electric Cargo Bike']

Parameter setup complete.

The parameter setup also translates the raw datasets into optimization-ready dictionaries, including facility fixed costs, facility capacities, zone demand, vehicle capacity, vehicle emissions, vehicle operating costs, low-emission zone indicators, travel times, route distances, and vehicle feasibility rules. These parameters connect the real business data to the mathematical model. From a business

perspective, this section creates the decision system’s “operating logic”: it defines what options are available, what they cost, and what operational restrictions apply.

1.6 7. LP baseline model

The LP model is a relaxed facility-location and allocation model. It allows fractional facility opening and continuous flow. Because it is not fully implementable, it is **not** used as the final recommendation. It is used to:

- confirm feasibility,
- provide a relaxed cost benchmark,
- identify binding demand and capacity constraints,
- support sensitivity interpretation using dual values.

```
[21]: # 7. LP baseline model

lp = Model("LP_Baseline_Relaxed")
lp.setParam("OutputFlag", 0)

x_lp = lp.addVars(facilities, zones, lb=0, vtype=GRB.CONTINUOUS, name="flow")
y_lp = lp.addVars(facilities, lb=0, ub=1, vtype=GRB.CONTINUOUS,
    ↪name="open_fraction")

# Use average vehicle operating cost for a clean LP baseline.
avg_vehicle_cost = float(np.mean(list(vehicle_op_cost.values())))
transport_lp = quicksum(avg_vehicle_cost * route_distance[i, j] * x_lp[i, j],
    ↪for i in facilities for j in zones)
fixed_lp = quicksum(facility_daily_fixed_cost[i] * y_lp[i] for i in facilities)
lp.setObjective(transport_lp + fixed_lp, GRB.MINIMIZE)

for j in zones:
    lp.addConstr(quicksum(x_lp[i, j] for i in facilities) == zone_demand[j],
    ↪name=f"demand_{j}")

for i in facilities:
    lp.addConstr(quicksum(x_lp[i, j] for j in zones) <= facility_capacity[i] *
    ↪y_lp[i], name=f"capacity_{i}")

lp.optimize()

if lp.status == GRB.OPTIMAL:
    lp_obj = lp.objVal
    print(f"LP baseline optimal daily cost: ${lp_obj:,.2f}")
    print("LP is feasible. This is a relaxed benchmark, not the final plan.")
else:
    raise RuntimeError(f"LP model did not solve to optimality. Status code: {lp.
    ↪status}")
```

Set parameter Username
Set parameter LicenseID to value 2829616
Academic license - for non-commercial use only - expires 2027-06-01
LP baseline optimal daily cost: \$1,640.35
LP is feasible. This is a relaxed benchmark, not the final plan.

The LP model provides a relaxed benchmark for the logistics network. It allows fractional facility opening and continuous package flows, so it is not meant to be the final implementable plan. Instead, it is used to test whether demand can be satisfied, estimate a lower-cost benchmark, and support sensitivity analysis through shadow prices.

The LP result shows that the network is feasible under relaxed assumptions. Because the LP allows fractional facility usage, its cost is lower than the more realistic C103 MIP model. This is expected. The LP should therefore be interpreted as a reference point, not a recommendation. It tells us what the network might achieve under idealized conditions before binary facility and assignment decisions are enforced.

```
[23]: # 7.1 LP sensitivity / shadow prices

lp_sensitivity = []
for j in zones:
    c = lp.getConstrByName(f"demand_{j}")
    lp_sensitivity.append({
        "constraint": f"demand_{j}",
        "zone": j,
        "shadow_price": c.Pi,
        "slack": c.Slack,
        "interpretation": "Marginal daily cost increase for one more package of_
↪demand in this zone."
    })

lp_sens_df = pd.DataFrame(lp_sensitivity).sort_values("shadow_price",
↪ascending=False)
display(lp_sens_df.head(10))

capacity_binding = []
for i in facilities:
    c = lp.getConstrByName(f"capacity_{i}")
    capacity_binding.append({
        "facility": i,
        "capacity_slack": c.Slack,
        "shadow_price": c.Pi,
        "open_fraction": y_lp[i].X,
        "used_capacity": sum(x_lp[i, j].X for j in zones),
        "max_capacity": facility_capacity[i],
    })
```

```
capacity_df = pd.DataFrame(capacity_binding).sort_values("capacity_slack")
display(capacity_df.head(10))
```

	constraint	zone	shadow_price	slack \
24	demand_Z025	Z025	0.328345	0.0
17	demand_Z018	Z018	0.316684	0.0
27	demand_Z028	Z028	0.279793	0.0
20	demand_Z021	Z021	0.274599	0.0
5	demand_Z006	Z006	0.260368	0.0
8	demand_Z009	Z009	0.243667	0.0
29	demand_Z030	Z030	0.239020	0.0
19	demand_Z020	Z020	0.229959	0.0
23	demand_Z024	Z024	0.226918	0.0
2	demand_Z003	Z003	0.226195	0.0

	interpretation
24	Marginal daily cost increase for one more pack...
17	Marginal daily cost increase for one more pack...
27	Marginal daily cost increase for one more pack...
20	Marginal daily cost increase for one more pack...
5	Marginal daily cost increase for one more pack...
8	Marginal daily cost increase for one more pack...
29	Marginal daily cost increase for one more pack...
19	Marginal daily cost increase for one more pack...
23	Marginal daily cost increase for one more pack...
2	Marginal daily cost increase for one more pack...

	facility	capacity_slack	shadow_price	open_fraction	used_capacity \
0	L001	0.0	-0.159269	0.070000	350.0
22	L023	0.0	-0.087268	0.000000	0.0
21	L022	0.0	-0.137851	0.045614	260.0
20	L021	0.0	-0.267174	0.000000	0.0
19	L020	0.0	-0.222534	0.000000	0.0
18	L019	0.0	-0.110242	0.070139	505.0
17	L018	0.0	-0.311734	0.000000	0.0
16	L017	0.0	-0.161092	0.172917	830.0
15	L016	0.0	-0.131218	0.092500	555.0
14	L015	0.0	-0.204145	0.000000	0.0

	max_capacity
0	5000
22	6800
21	5700
20	2000
19	3500
18	7200
17	2400
16	4800

15	6000
14	3600

The shadow price analysis identifies which demand zones are most cost-sensitive. Demand constraints have zero slack, meaning demand is fully binding across zones. Zones with higher shadow prices, such as Z025, Z018, and Z028, have the largest marginal cost impact if demand increases by one package. Capacity shadow prices are negative because of the minimization sign convention; more negative values suggest that additional facility capacity would reduce cost in the LP relaxation.

```
[25]: # 7.2 LP heatmap dashboard

if 'lp' in globals() and lp.status == GRB.OPTIMAL:
    flow_matrix = np.array([[x_lp[i, j].X for j in zones] for i in facilities])
    y_vals = np.array([y_lp[i].X for i in facilities])
    cap_used = flow_matrix.sum(axis=1)
    cap_total = np.array([facility_capacity[i] for i in facilities])
    util_pct = np.divide(cap_used, cap_total, out=np.zeros_like(cap_used,
    dtype=float), where=cap_total != 0) * 100

    demand_duals = np.array([lp.getConstrByName(f'demand_{j}').Pi for j in
    zones])
    cap_duals = np.array([lp.getConstrByName(f'capacity_{i}').Pi for i in
    facilities])

    fig = plt.figure(figsize=(20, 14))
    fig.patch.set_facecolor('#0f1117')
    fig.suptitle('LP Base Model - Heatmap Dashboard',
                 fontsize=16, color=PALETTE['teal'], fontweight='bold', y=0.98)
    gs = fig.add_gridspec(2, 2, hspace=0.45, wspace=0.35)

    # Flow allocation heatmap
    ax1 = fig.add_subplot(gs[0, :])
    ax1.set_facecolor('#1a1d2e')
    im1 = ax1.imshow(flow_matrix, cmap='YlOrRd', aspect='auto', vmin=0)
    ax1.set_xticks(range(len(zones)))
    ax1.set_xticklabels(zones, rotation=55, ha='right', fontsize=8,
    color='#8892b0')
    ax1.set_yticks(range(len(facilities)))
    ax1.set_yticklabels(facilities, fontsize=9, color='#c8d0e7')
    ax1.set_title('LP Flow Allocation x (packages/day)', color='#c8d0e7',
    pad=10, fontsize=11)
    ax1.set_xlabel('Demand Zone', color='#8892b0')
    ax1.set_ylabel('Facility', color='#8892b0')
    for ri, i in enumerate(facilities):
        for ci, j in enumerate(zones):
            val = flow_matrix[ri, ci]
```



```

        if val > 1e-6:
            ax1.text(ci, ri, f'{val:.0f}', ha='center', va='center',
↪fontsize=7,
                                color='#0f1117' if val > flow_matrix.max() * 0.55 else
↪'white')
        cb1 = fig.colorbar(im1, ax=ax1, fraction=0.02, pad=0.01)
        cb1.set_label('Packages / Day', color='#c8d0e7', fontsize=9)

        # Facility openness and utilization
        ax2 = fig.add_subplot(gs[1, 0])
        ax2.set_facecolor('#1a1d2e')
        util_matrix = np.vstack([y_vals, util_pct / 100])
        im2 = ax2.imshow(util_matrix, cmap='plasma', aspect='auto', vmin=0, vmax=1)
        ax2.set_xticks(range(len(facilities)))
        ax2.set_xticklabels(facilities, rotation=40, ha='right', fontsize=8,
↪color='#8892b0')
        ax2.set_yticks([0, 1])
        ax2.set_yticklabels(['y open fraction', 'Utilization %'], fontsize=8,
↪color='#c8d0e7')
        ax2.set_title('Facility Openness & Utilization', color='#c8d0e7', pad=10,
↪fontsize=11)
        for ci, i in enumerate(facilities):
            ax2.text(ci, 0, f'{y_vals[ci]:.2f}', ha='center', va='center',
↪fontsize=8,
                                color='white' if y_vals[ci] < 0.6 else '#0f1117')
            ax2.text(ci, 1, f'{util_pct[ci]:.0f}%', ha='center', va='center',
↪fontsize=8,
                                color='white' if util_pct[ci] < 60 else '#0f1117')
        fig.colorbar(im2, ax=ax2, fraction=0.05, pad=0.02).set_label('0 → 1 scale',
↪color='#c8d0e7')

        # Dual values
        ax3 = fig.add_subplot(gs[1, 1])
        ax3.set_facecolor('#1a1d2e')
        dual_data = np.vstack([
            demand_duals,
            np.pad(cap_duals, (0, max(0, len(zones) - len(cap_duals))),
↪constant_values=np.nan)[:len(zones)]
        ])
        im3 = ax3.imshow(dual_data, cmap='coolwarm', aspect='auto')
        ax3.set_xticks(range(len(zones)))
        ax3.set_xticklabels(zones, rotation=55, ha='right', fontsize=8,
↪color='#8892b0')
        ax3.set_yticks([0, 1])
        ax3.set_yticklabels(['Demand shadow prices', 'Capacity duals (padded)'],
↪fontsize=8, color='#c8d0e7')

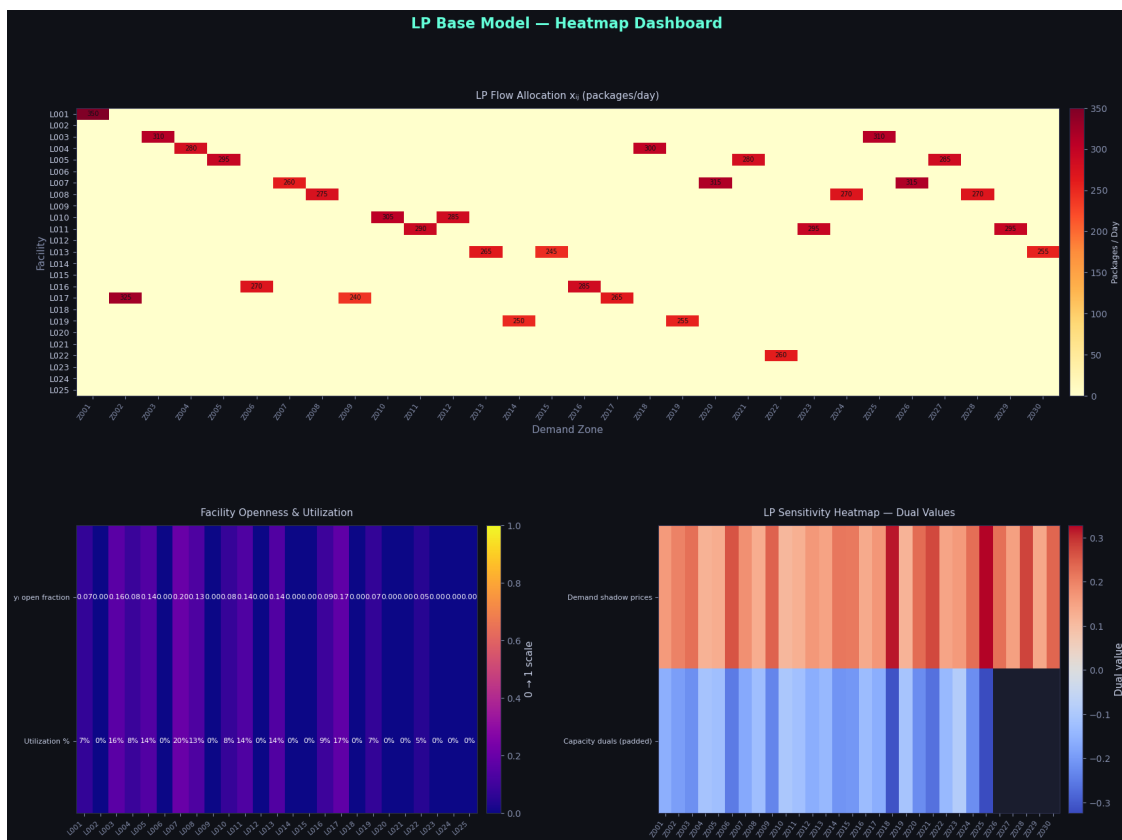
```

```

ax3.set_title('LP Sensitivity Heatmap - Dual Values', color='#c8d0e7',
pad=10, fontsize=11)
fig.colorbar(im3, ax=ax3, fraction=0.05, pad=0.02).set_label('Dual value',
color='#c8d0e7')

plt.savefig('lp_heatmap_dashboard_original_style.png', dpi=150,
bbox_inches='tight', facecolor='#0f1117')
plt.show()
print('LP heatmap dashboard rendered.')
else:
    print('LP model has not solved optimally yet; run Section 7 before this
visualization.')

```



LP heatmap dashboard rendered.

The LP heatmap dashboard further supports interpretation by showing how packages are allocated across facilities and zones, which facilities are partially opened in the relaxed solution, and which constraints have high dual values. These results support sensitivity insight, but the final recommendation still comes from the C103 MIP because the MIP is operationally realistic.

1.7 8. Main C103 mixed-integer programming model

This is the primary model for the final recommendation.

1.7.1 Decision variables

- $y[i] = 1$ if facility i is opened.
- $z[i, j] = 1$ if facility i serves zone j .
- $w[i, j, v] = 1$ if route $i \rightarrow j$ uses vehicle type v .
- $x[i, j, v] =$ packages delivered from facility i to zone j using vehicle v .
- $n[i, j, v] =$ number of daily trips on route $i \rightarrow j$ using vehicle v .

1.7.2 Important C103 constraints

- Each demand zone is assigned to exactly one facility.
- Each assigned zone is served completely.
- Each open facility serves at most 6 zones, representing the 6-stop route requirement.
- Low-emission zones must use electric vehicles.
- A vehicle assignment is feasible only if travel time is within 120 minutes and vehicle range is sufficient.

```
[28]: # 8. Helper function for the full C103 model

def route_allowed(i, j, v, max_window_mins=MAX_DELIVERY_WINDOW_MINS,
    min_reliability=MIN_TIME_RELIABILITY):
    '''Return True if vehicle v can legally and operationally serve route i ->
    j.'''
    if (i, j) not in route_distance:
        return False
    if route_time[i, j] > max_window_mins:
        return False
    if route_reliability[i, j] < min_reliability:
        return False
    # Round trip distance must be within vehicle range.
    if 2 * route_distance[i, j] > vehicle_range[v]:
        return False
    # C103: low-emission zones require electric vehicles.
    if j in low_emission_zones and v not in electric_vehicles:
        return False
    return True

allowed_arcs = [(i, j, v) for i in facilities for j in zones for v in vehicles
    if route_allowed(i, j, v)]
allowed_by_ij = {(i, j): [v for v in vehicles if route_allowed(i, j, v)] for i
    in facilities for j in zones}

# Check whether every zone has at least one feasible facility-vehicle
assignment.
```

```

no_feasible_zones = [j for j in zones if sum(len(allowed_by_ij[i, j]) for i in
    ↪ facilities) == 0]
if no_feasible_zones:
    raise ValueError(f"These zones have no feasible C103 route/vehicle_
    ↪ assignment: {no_feasible_zones}")

print(f"Feasible facility-zone-vehicle assignments: {len(allowed_arcs):,}")
print("All zones have at least one feasible C103 assignment.")

```

Feasible facility-zone-vehicle assignments: 5,100
 All zones have at least one feasible C103 assignment.

The C103 MIP is the primary model for the final recommendation because it directly represents real operating decisions. Unlike the LP model, this model includes binary decisions for whether a facility is opened, whether a facility serves a demand zone, and which vehicle type is used on each route. These binary variables make the model much closer to an implementable logistics plan.

The model also directly incorporates the C103 client requirements. Each selected zone is served completely rather than partially. Each zone is assigned to one facility and one vehicle type. Each open facility can serve at most 6 zones, which approximates the client's 6-stop route limit. Low-emission zones are restricted to electric vehicles, and route assignments are allowed only when the travel time is within the 2-hour window and vehicle range is sufficient.

[30]: *# 8.1 Solve the primary C103 cost-minimizing MIP*

```

def solve_c103_model(
    objective="cost",
    emission_limit=None,
    cost_limit=None,
    max_facilities=None,
    min_service_ratio=1.0,
    fixed_open=None,
    fixed_assignments=None,
    fuel_cost_multiplier=1.0,
    emission_weight=0.0,
    service_penalty=5000,
    time_limit=180,
    mip_gap=0.01,
    verbose=False,
):
    '''Build and solve the C103 MIP.

    objective options:
    - cost: minimize daily cost
    - emissions: minimize daily emissions
    - weighted: minimize cost + emission_weight * emissions - service reward

```

```

- service: maximize served demand, then use penalties through objective
↪expression
'''
m = Model(f"C103_{objective}")
m.setParam("OutputFlag", 1 if verbose else 0)
m.setParam("TimeLimit", time_limit)
m.setParam("MIPGap", mip_gap)

y = m.addVars(facilities, vtype=GRB.BINARY, name="open_facility")
served = m.addVars(zones, vtype=GRB.BINARY, name="served_zone")
z = m.addVars(facilities, zones, vtype=GRB.BINARY,
↪name="assign_facility_zone")
w = m.addVars(allowed_arcs, vtype=GRB.BINARY, name="assign_vehicle")
x = m.addVars(allowed_arcs, lb=0, vtype=GRB.CONTINUOUS, name="packages")
n = m.addVars(allowed_arcs, lb=0, vtype=GRB.INTEGER, name="trips")

# Demand service: completely served or not served. Main model sets
↪min_service_ratio=1.
total_demand = sum(zone_demand[j] for j in zones)
for j in zones:
    m.addConstr(quicksum(x[i, j, v] for i in facilities for v in
↪allowed_by_ij[i, j]) == zone_demand[j] * served[j],
↪name=f"complete_service_{j}")
    m.addConstr(quicksum(z[i, j] for i in facilities) == served[j],
↪name=f"one_facility_if_served_{j}")

    m.addConstr(quicksum(zone_demand[j] * served[j] for j in zones) >=
↪min_service_ratio * total_demand, name="minimum_service_ratio")

# Facility capacity and stop limits.
for i in facilities:
    m.addConstr(quicksum(x[i, j, v] for j in zones for v in
↪allowed_by_ij[i, j]) <= facility_capacity[i] * y[i],
↪name=f"facility_capacity_{i}")
    m.addConstr(quicksum(z[i, j] for j in zones) <= MAX_STOPS_PER_ROUTE *
↪y[i], name=f"six_stop_limit_{i}")
    if fixed_open is not None:
        m.addConstr(y[i] == (1 if i in fixed_open else 0),
↪name=f"fixed_open_{i}")

    if max_facilities is not None:
        m.addConstr(quicksum(y[i] for i in facilities) <= max_facilities,
↪name="max_facilities")

# Assignment, vehicle choice, and trip capacity.

```

```

max_trips_needed = int(math.ceil(max(zone_demand.values()) /
↳max(min(vehicle_capacity.values(), 1))) + 2
    for i in facilities:
        for j in zones:
            allowed_vs = allowed_by_ij[i, j]
            if not allowed_vs:
                m.addConstr(z[i, j] == 0,
↳name=f"no_feasible_assignment_{i}_{j}")
                continue
            m.addConstr(z[i, j] <= y[i], name=f"assign_only_if_open_{i}_{j}")
            m.addConstr(quicksum(w[i, j, v] for v in allowed_vs) == z[i, j],
↳name=f"one_vehicle_if_assigned_{i}_{j}")
            if fixed_assignments is not None:
                m.addConstr(z[i, j] == (1 if fixed_assignments.get(j) == i else
↳0), name=f"fixed_assignment_{i}_{j}")
            for v in allowed_vs:
                m.addConstr(x[i, j, v] <= zone_demand[j] * w[i, j, v],
↳name=f"flow_link_{i}_{j}_{v}")
                m.addConstr(x[i, j, v] <= vehicle_capacity[v] * n[i, j, v],
↳name=f"vehicle_capacity_{i}_{j}_{v}")
                m.addConstr(n[i, j, v] >= w[i, j, v],
↳name=f"trip_if_used_{i}_{j}_{v}")
                m.addConstr(n[i, j, v] <= max_trips_needed * w[i, j, v],
↳name=f"trip_link_{i}_{j}_{v}")

    # Optional environmental caps by zone, if available. These are kept soft
↳enough to avoid false infeasibility from approximate data.
    for j in zones:
        if j in zone_emission_cap and pd.notna(zone_emission_cap[j]) and
↳zone_emission_cap[j] > 0:
            zone_emis = quicksum(
                vehicle_emission[v] * route_distance[i, j] *
↳route_emission_factor[i, j] * n[i, j, v]
                for i in facilities for v in allowed_by_ij[i, j]
            )
            # Do not enforce unrealistically tiny caps caused by synthetic/
↳approximate data.
            if zone_emission_cap[j] >= 0.01:
                m.addConstr(zone_emis <= zone_emission_cap[j] + 1000 * (1 -
↳served[j]), name=f"zone_emission_cap_{j}")

    # Objective components.
    fixed_cost = quicksum(facility_daily_fixed_cost[i] * y[i] for i in
↳facilities)
    trip_cost = quicksum(

```

```

        (vehicle_op_cost[v] * fuel_cost_multiplier * route_distance[i, j] +
vehicle_daily_fixed_cost[v]) * n[i, j, v]
        for (i, j, v) in allowed_arcs
    )
    cost_expr = fixed_cost + trip_cost

    emission_expr = quicksum(
        vehicle_emission[v] * route_distance[i, j] * route_emission_factor[i,
j] * n[i, j, v]
        for (i, j, v) in allowed_arcs
    )
    service_expr = quicksum(zone_demand[j] * served[j] for j in zones)

    if emission_limit is not None:
        m.addConstr(emission_expr <= emission_limit,
name="epsilon_emission_limit")
    if cost_limit is not None:
        m.addConstr(cost_expr <= cost_limit, name="cost_limit")

    if objective == "cost":
        m.setObjective(cost_expr + service_penalty * (total_demand -
service_expr), GRB.MINIMIZE)
    elif objective == "emissions":
        m.setObjective(emission_expr + 0.0001 * cost_expr + service_penalty *
(total_demand - service_expr), GRB.MINIMIZE)
    elif objective == "weighted":
        m.setObjective(cost_expr + emission_weight * emission_expr +
service_penalty * (total_demand - service_expr), GRB.MINIMIZE)
    elif objective == "service":
        m.setObjective(-service_expr + 0.000001 * cost_expr, GRB.MINIMIZE)
    else:
        raise ValueError("Unknown objective type.")

m.optimize()

result = {
    "model": m,
    "y": y,
    "served": served,
    "z": z,
    "w": w,
    "x": x,
    "n": n,
    "cost_expr": cost_expr,
    "emission_expr": emission_expr,
    "service_expr": service_expr,

```

```

        "status": m.status,
    }

    if m.status in [GRB.OPTIMAL, GRB.TIME_LIMIT, GRB.SUBOPTIMAL] and m.SolCount > 0:
        result.update({
            "objective_value": m.objVal,
            "cost": cost_expr.getValue(),
            "emissions": emission_expr.getValue(),
            "service": service_expr.getValue(),
            "open_facilities": [i for i in facilities if y[i].X > 0.5],
            "served_zones": [j for j in zones if served[j].X > 0.5],
            "assignments": {j: next((i for i in facilities if z[i, j].X > 0.5),
                                   None) for j in zones if served[j].X > 0.5},
            "vehicle_assignments": {
                (i, j): next((v for v in allowed_by_ij[i, j] if (i, j, v) in
                             allowed_arcs and w[i, j, v].X > 0.5), None)
                for i in facilities for j in zones if z[i, j].X > 0.5
            },
        })
    return result

c103_cost = solve_c103_model(objective="cost", min_service_ratio=1.0,
                             time_limit=180, mip_gap=0.01)

if c103_cost["status"] not in [GRB.OPTIMAL, GRB.TIME_LIMIT, GRB.SUBOPTIMAL] or
   c103_cost["model"].SolCount == 0:
    raise RuntimeError(f"C103 MIP failed. Status code: {c103_cost['status']}")

print("C103 MIP solved.")
print(f"Daily cost: ${c103_cost['cost']:.2f}")
print(f"Daily emissions: {c103_cost['emissions']:.2f} kg CO2")
print(f"Served demand: {c103_cost['service']:.0f} packages/day")
print(f"Open facilities: {c103_cost['open_facilities']}")

```

C103 MIP solved.

Daily cost: \$4,939.40

Daily emissions: 1.42 kg CO2

Served demand: 8,500 packages/day

Open facilities: ['L005', 'L007', 'L013', 'L017', 'L024']

The final cost-minimizing C103 solution selects 5 facilities: L005, L007, L013, L017, and L024. This result is meaningful because 5 facilities is also the minimum number needed to serve 30 zones under a 6-zone-per-facility limit. The model serves all 30 zones and all 8,500 daily packages while using electric delivery trucks for the recommended assignments. This satisfies the C103 client requirements and provides a practical operating plan.

[32]: # 8.2 Display C103 recommendation

```

solution_rows = []
for j, i in c103_cost["assignments"].items():
    v = c103_cost["vehicle_assignments"].get((i, j))
    trips = c103_cost["n"][i, j, v].X if v is not None else np.nan
    solution_rows.append({
        "zone": j,
        "district": zone_district.get(j, "Unknown"),
        "demand": zone_demand[j],
        "assigned_facility": i,
        "vehicle": v,
        "trips_per_day": int(round(trips)) if not pd.isna(trips) else None,
        "distance_km": route_distance[i, j],
        "travel_time_mins": route_time[i, j],
        "low_emission_zone": "Yes" if j in low_emission_zones else "No",
    })

c103_solution_df = pd.DataFrame(solution_rows).
    ↪sort_values(["assigned_facility", "zone"])
display(c103_solution_df)

facility_summary = (
    c103_solution_df.groupby("assigned_facility")
    .agg(
        zones_served=("zone", "count"),
        packages_served=("demand", "sum"),
        avg_distance_km=("distance_km", "mean"),
        avg_travel_time_mins=("travel_time_mins", "mean"),
    )
    .reset_index()
)
facility_summary["capacity"] = facility_summary["assigned_facility"].
    ↪map(facility_capacity)
facility_summary["utilization_pct"] = 100 * facility_summary["packages_served"]_
    ↪ / facility_summary["capacity"]
display(facility_summary)

```

	zone		district	demand	assigned_facility	vehicle	\
3	Z004		East Side	280	L005	Delivery Truck - Electric	
4	Z005		East Side	295	L005	Delivery Truck - Electric	
10	Z011	South District		290	L005	Delivery Truck - Electric	
17	Z018	South District		300	L005	Delivery Truck - Electric	
20	Z021		East Side	280	L005	Delivery Truck - Electric	
26	Z027		East Side	285	L005	Delivery Truck - Electric	
0	Z001		Downtown	350	L007	Delivery Truck - Electric	

2	Z003	Downtown	310	L007	Delivery Truck - Electric
6	Z007	West Side	260	L007	Delivery Truck - Electric
8	Z009	West Side	240	L007	Delivery Truck - Electric
24	Z025	Central	310	L007	Delivery Truck - Electric
25	Z026	Central	315	L007	Delivery Truck - Electric
12	Z013	North District	265	L013	Delivery Truck - Electric
13	Z014	North District	250	L013	Delivery Truck - Electric
14	Z015	North District	245	L013	Delivery Truck - Electric
15	Z016	East Side	285	L013	Delivery Truck - Electric
18	Z019	North District	255	L013	Delivery Truck - Electric
29	Z030	North District	255	L013	Delivery Truck - Electric
1	Z002	Downtown	325	L017	Delivery Truck - Electric
9	Z010	South District	305	L017	Delivery Truck - Electric
11	Z012	South District	285	L017	Delivery Truck - Electric
16	Z017	West Side	265	L017	Delivery Truck - Electric
22	Z023	South District	295	L017	Delivery Truck - Electric
28	Z029	South District	295	L017	Delivery Truck - Electric
5	Z006	East Side	270	L024	Delivery Truck - Electric
7	Z008	West Side	275	L024	Delivery Truck - Electric
19	Z020	Central	315	L024	Delivery Truck - Electric
21	Z022	North District	260	L024	Delivery Truck - Electric
23	Z024	West Side	270	L024	Delivery Truck - Electric
27	Z028	West Side	270	L024	Delivery Truck - Electric

	trips_per_day	distance_km	travel_time_mins	low_emission_zone
3	1	0.336	0.8	Yes
4	1	0.028	0.1	Yes
10	1	1.219	2.9	No
17	1	0.874	2.1	No
20	1	0.666	1.6	Yes
26	1	0.131	0.3	Yes
0	1	0.920	2.2	Yes
2	1	0.552	1.3	Yes
6	1	0.033	0.1	Yes
8	1	0.344	0.8	Yes
24	1	0.883	2.1	Yes
25	1	0.237	0.6	Yes
12	1	0.033	0.1	Yes
13	1	0.531	1.3	Yes
14	1	0.316	0.8	Yes
15	1	1.332	3.2	Yes
18	1	1.022	2.5	Yes
29	1	0.434	1.0	Yes
1	1	0.177	0.4	Yes
9	1	0.646	1.6	No
11	1	0.923	2.2	No
16	1	0.033	0.1	Yes
22	1	1.301	3.1	No

28	1	1.143	2.7	No
5	1	1.590	3.8	Yes
7	1	0.431	1.0	Yes
19	1	0.420	1.0	Yes
21	1	0.896	2.2	Yes
23	1	0.033	0.1	Yes
27	1	0.322	0.8	Yes

	assigned_facility	zones_served	packages_served	avg_distance_km	\
0	L005	6	1730	0.542333	
1	L007	6	1785	0.494833	
2	L013	6	1555	0.611333	
3	L017	6	1770	0.703833	
4	L024	6	1660	0.615333	

	avg_travel_time_mins	capacity	utilization_pct
0	1.300000	6000	28.833333
1	1.183333	4500	39.666667
2	1.483333	5500	28.272727
3	1.683333	4800	36.875000
4	1.483333	3200	51.875000

From a business perspective, the C103 MIP balances cost control with service feasibility and sustainability compliance. It is more expensive than the LP relaxation, but it is more realistic because it respects facility-opening logic, vehicle assignment, stop limits, low-emission rules, and delivery-time feasibility.

[34]: *# 8.3 Verify C103 requirements explicitly*

```

checks = []

# 1. At most 6 zones per open facility.
stops_ok = all(facility_summary["zones_served"] <= MAX_STOPS_PER_ROUTE)
checks.append({"requirement": "At most 6 stops per planned route/facility",
    ↪ "passed": stops_ok})

# 2. Electric vehicles in low-emission zones.
lez_rows = c103_solution_df[c103_solution_df["low_emission_zone"] == "Yes"]
lez_ok = all(lez_rows["vehicle"].isin(electric_vehicles)) if len(lez_rows) else
    ↪ True
checks.append({"requirement": "Electric vehicles in low-emission zones",
    ↪ "passed": lez_ok})

# 3. Delivery within 2-hour windows.
time_ok = all(c103_solution_df["travel_time_mins"] <= MAX_DELIVERY_WINDOW_MINS)
checks.append({"requirement": "Travel time <= 120 minutes", "passed": time_ok})

```

```

# 4. Complete service.
complete_ok = c103_solution_df["demand"].sum() == sum(zone_demand.values())
checks.append({"requirement": "All zones served completely", "passed":
    ↪complete_ok})

# 5. One facility per zone.
one_facility_ok = c103_solution_df["zone"].nunique() == len(zones)
checks.append({"requirement": "Each zone assigned to exactly one facility",
    ↪"passed": one_facility_ok})

checks_df = pd.DataFrame(checks)
display(checks_df)

if checks_df["passed"].all():
    print("All C103 requirement checks passed.")
else:
    print("One or more C103 checks failed. Review the table above.")

```

	requirement	passed
0	At most 6 stops per planned route/facility	True
1	Electric vehicles in low-emission zones	True
2	Travel time <= 120 minutes	True
3	All zones served completely	True
4	Each zone assigned to exactly one facility	True

All C103 requirement checks passed.

The C103 verification table confirms that the final recommended solution satisfies the client-specific operating rules. The model passes the 6-stop requirement because each open facility serves no more than 6 zones. It passes the electric vehicle requirement because low-emission zones are assigned only to electric vehicles. It passes the delivery-time requirement because all selected routes fall within the 120-minute maximum travel time. It also satisfies the complete-service logic because each served zone receives its full demand from exactly one facility and one vehicle type.

[36]: # 8.4 MIP Results Dashboard

```

if 'c103_solution_df' in globals() and len(c103_solution_df) > 0:
    fig, axes = plt.subplots(1, 2, figsize=(16, 6))
    fig.patch.set_facecolor('#0f1117')
    fig.suptitle('C103 MIP Results - Facility Utilization and Vehicle Mix',
        fontsize=15, color=PALETTE['teal'], fontweight='bold')

    # Facility utilization
    ax1 = axes[0]
    ax1.set_facecolor('#1a1d2e')
    util_df = facility_summary.sort_values('utilization_pct', ascending=True)
    util_colors = [PALETTE['green'] if u < 80 else PALETTE['orange'] if u < 95
    ↪else PALETTE['red']]

```

```

        for u in util_df['utilization_pct']
        ax1.barh(util_df['assigned_facility'], util_df['utilization_pct'],
        color=util_colors, edgecolor='#0f1117')
        ax1.axvline(100, color=PALETTE['red'], linestyle='--', linewidth=1.5,
        label='Capacity limit')
        ax1.set_title('Selected Facility Utilization', color='#c8d0e7', pad=10)
        ax1.set_xlabel('Utilization (%)')
        ax1.set_ylabel('Facility')
        ax1.legend(facecolor='#1a1d2e', edgecolor='#2e3250', labelcolor='#c8d0e7')
        ax1.grid(True, axis='x')
        for _, row in util_df.iterrows():
            ax1.text(row['utilization_pct'] + 1, row['assigned_facility'],
            f"{row['utilization_pct']:.0f}%",
            va='center', fontsize=8, color='#c8d0e7')

    # Vehicle mix
    ax2 = axes[1]
    ax2.set_facecolor('#1a1d2e')
    vehicle_mix = c103_solution_df.groupby('vehicle')['demand'].sum().
    sort_values(ascending=True)
    mix_colors = [PALETTE['green'] if 'Electric' in str(v) or v in
    electric_vehicles else PALETTE['orange']
    for v in vehicle_mix.index]
    ax2.barh(vehicle_mix.index, vehicle_mix.values, color=mix_colors,
    edgecolor='#0f1117')
    ax2.set_title('C103 Vehicle Usage by Packages Served', color='#c8d0e7',
    pad=10)
    ax2.set_xlabel('Packages / Day')
    ax2.set_ylabel('Vehicle Type')
    ax2.grid(True, axis='x')

    plt.tight_layout()
    plt.savefig('c103_results_original_style.png', dpi=150,
    bbox_inches='tight', facecolor='#0f1117')
    plt.show()

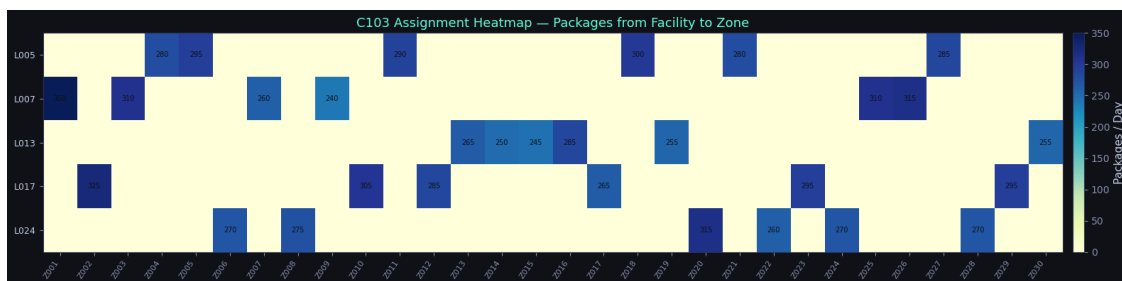
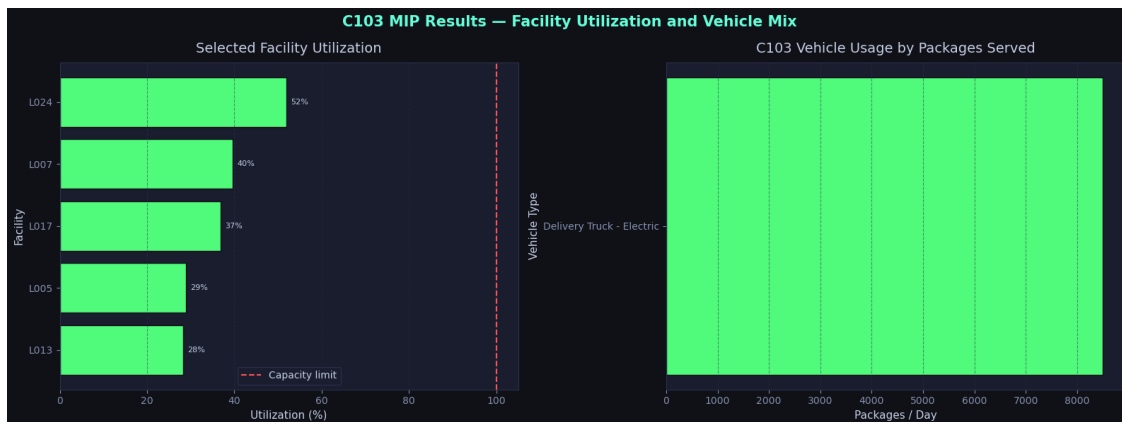
    # Assignment heatmap: selected facility by zone
    assign_matrix = pd.crosstab(c103_solution_df['assigned_facility'],
    c103_solution_df['zone'], values=c103_solution_df['demand'], aggfunc='sum').
    fillna(0)
    fig, ax = plt.subplots(figsize=(16, max(4, 0.4 * len(assign_matrix))))
    fig.patch.set_facecolor('#0f1117')
    ax.set_facecolor('#1a1d2e')
    im = ax.imshow(assign_matrix.values, cmap='YlGnBu', aspect='auto')
    ax.set_xticks(range(assign_matrix.shape[1]))

```

```

    ax.set_xticklabels(assign_matrix.columns, rotation=55, ha='right',
↳ fontsize=8, color='#8892b0')
    ax.set_yticks(range(assign_matrix.shape[0]))
    ax.set_yticklabels(assign_matrix.index, fontsize=9, color='#c8d0e7')
    ax.set_title('C103 Assignment Heatmap - Packages from Facility to Zone',
↳ color=PALETTE['teal'], fontsize=13)
    for r in range(assign_matrix.shape[0]):
        for c in range(assign_matrix.shape[1]):
            val = assign_matrix.values[r, c]
            if val > 0:
                ax.text(c, r, f'{val:.0f}', ha='center', va='center',
↳ fontsize=7, color='#0f1117')
    fig.colorbar(im, ax=ax, fraction=0.025, pad=0.01).set_label('Packages /
↳ Day', color='#c8d0e7')
    plt.tight_layout()
    plt.savefig('c103_assignment_heatmap_original_style.png', dpi=150,
↳ bbox_inches='tight', facecolor='#0f1117')
    plt.show()
    print('C103 result visualizations rendered.')
else:
    print('C103 solution table not found; run Section 8 before this
↳ visualization.')

```



C103 result visualizations rendered.

```
[37]: # =====
# Delivery Time Workload Visualization
# =====
# Uses c103_solution_df columns:
# ['zone', 'district', 'demand', 'assigned_facility', 'vehicle',
#  'trips_per_day', 'distance_km', 'travel_time_mins', 'low_emission_zone']
#
# Interpretation:
# This estimates total route-time workload by open facility.
# Since the model does not sequence multi-stop routes, we approximate:
# total daily driving time = 2 * travel_time_mins * trips_per_day
#
# The factor of 2 approximates outbound travel plus return-to-facility travel.

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

if "c103_solution_df" not in globals():
    raise NameError("c103_solution_df not found. Run the C103 MIP solution_
↪section first.")

required_cols = [
    "zone", "demand", "assigned_facility", "vehicle",
    "trips_per_day", "distance_km", "travel_time_mins"
]

missing_cols = [c for c in required_cols if c not in c103_solution_df.columns]
if missing_cols:
    raise KeyError(f"Missing required columns in c103_solution_df:
↪{missing_cols}")

delivery_time_df = c103_solution_df.copy()

# Estimated daily driving workload per assigned zone.
delivery_time_df["one_way_daily_time_mins"] = (
    delivery_time_df["travel_time_mins"] * delivery_time_df["trips_per_day"]
)

delivery_time_df["round_trip_daily_time_mins"] = (
    2 * delivery_time_df["one_way_daily_time_mins"]
)

delivery_time_df["round_trip_daily_time_hours"] = (
```

```

    delivery_time_df["round_trip_daily_time_mins"] / 60
)

# Facility-level summary.
facility_time_df = (
    delivery_time_df
    .groupby("assigned_facility")
    .agg(
        stops=("zone", "nunique"),
        demand_served=("demand", "sum"),
        total_trips=("trips_per_day", "sum"),
        avg_one_way_time_per_stop=("travel_time_mins", "mean"),
        one_way_daily_time_mins=("one_way_daily_time_mins", "sum"),
        round_trip_daily_time_mins=("round_trip_daily_time_mins", "sum"),
        round_trip_daily_time_hours=("round_trip_daily_time_hours", "sum"),
    )
    .reset_index()
    .sort_values("round_trip_daily_time_mins", ascending=False)
)

total_round_trip_mins = facility_time_df["round_trip_daily_time_mins"].sum()
total_round_trip_hours = facility_time_df["round_trip_daily_time_hours"].sum()

print("Estimated C103 Delivery Time Workload")
print("=" * 45)
print(f"Total estimated round-trip driving time: {total_round_trip_mins:.1f}␣
↪minutes")
print(f"Total estimated round-trip driving time: {total_round_trip_hours:.2f}␣
↪hours")
print()
display(facility_time_df)

# -----
# Visualization
# -----

fig, axes = plt.subplots(1, 2, figsize=(17, 6))
fig.patch.set_facecolor("#0f1117")
fig.suptitle(
    "C103 Delivery Time Workload by Open Facility",
    fontsize=15,
    color=PALETTE["teal"],
    fontweight="bold",
    y=1.03,
)

# Plot 1: total estimated route time by facility

```



```

ax1 = axes[0]
ax1.set_facecolor("#1a1d2e")

plot_df = facility_time_df.sort_values("round_trip_daily_time_mins",
    ↪ascending=True)

ax1.barh(
    plot_df["assigned_facility"],
    plot_df["round_trip_daily_time_mins"],
    color=PALETTE["teal"],
    edgecolor="#0f1117",
)

for _, row in plot_df.iterrows():
    ax1.text(
        row["round_trip_daily_time_mins"] +
    ↪max(plot_df["round_trip_daily_time_mins"]) * 0.01,
        row["assigned_facility"],
        f'{row["round_trip_daily_time_mins"]:.1f} min',
        va="center",
        color="#c8d0e7",
        fontsize=9,
        fontweight="bold",
    )

ax1.set_title("Estimated Daily Driving Time", color="#c8d0e7", pad=10)
ax1.set_xlabel("Round-trip driving time (minutes)")
ax1.set_ylabel("Open facility")
ax1.grid(True, axis="x")

# Plot 2: trips and stops by facility
ax2 = axes[1]
ax2.set_facecolor("#1a1d2e")

x = np.arange(len(facility_time_df["assigned_facility"]))
width = 0.35

ax2.bar(
    x - width / 2,
    facility_time_df["stops"],
    width,
    label="Stops served",
    color=PALETTE["pink"],
    edgecolor="#0f1117",
)

ax2.bar(

```

```

        x + width / 2,
        facility_time_df["total_trips"],
        width,
        label="Total trips/day",
        color=PALETTE["orange"],
        edgecolor="#0f1117",
    )

ax2.set_xticks(x)
ax2.set_xticklabels(facility_time_df["assigned_facility"])
ax2.set_title("Stops and Trips by Facility", color="#c8d0e7", pad=10)
ax2.set_xlabel("Open facility")
ax2.set_ylabel("Count")
ax2.legend(facecolor="#1a1d2e", edgecolor="#2e3250", labelcolor="#c8d0e7")
ax2.grid(True, axis="y")

plt.tight_layout()
plt.savefig(
    "c103_delivery_time_workload.png",
    dpi=150,
    bbox_inches="tight",
    facecolor="#0f1117",
)
plt.show()

print("Delivery time workload visualization rendered.")

```

Estimated C103 Delivery Time Workload

=====

Total estimated round-trip driving time: 85.6 minutes

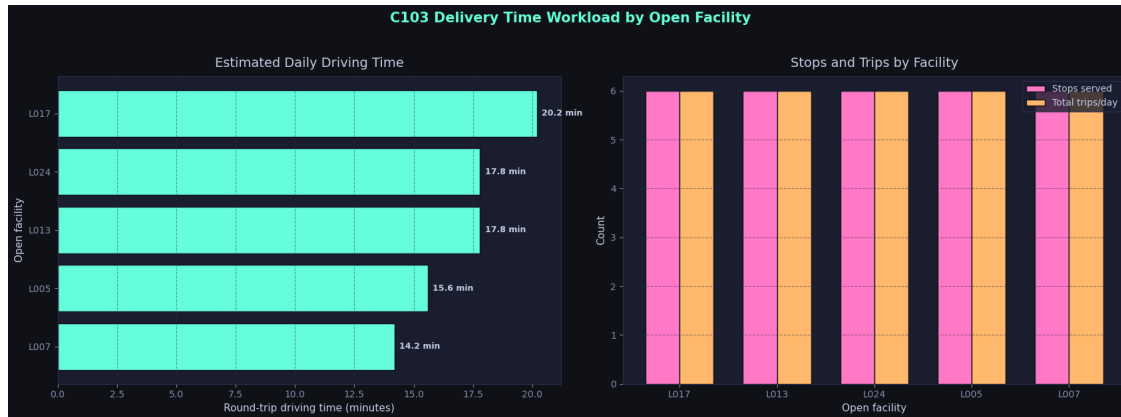
Total estimated round-trip driving time: 1.43 hours

	assigned_facility	stops	demand_served	total_trips \
3	L017	6	1770	6
2	L013	6	1555	6
4	L024	6	1660	6
0	L005	6	1730	6
1	L007	6	1785	6

	avg_one_way_time_per_stop	one_way_daily_time_mins \
3	1.683333	10.1
2	1.483333	8.9
4	1.483333	8.9
0	1.300000	7.8
1	1.183333	7.1

	round_trip_daily_time_mins	round_trip_daily_time_hours
--	----------------------------	-----------------------------

3	20.2	0.336667
2	17.8	0.296667
4	17.8	0.296667
0	15.6	0.260000
1	14.2	0.236667



Delivery time workload visualization rendered.

```
[38]: # =====
# Approximate Multi-Stop Route Visualization for C103
# =====
# This does NOT re-optimize the MIP.
# It creates an approximate route sequence after the model assigns zones to
# facilities.
#
# Assumption:
# Each open facility has one planned route visiting its assigned zones.
# Route order is estimated using a nearest-neighbor heuristic:
# start at facility -> visit closest unvisited zone -> repeat -> return to
# facility.

import math
import pandas as pd
import matplotlib.pyplot as plt

def euclidean_distance(lat1, lon1, lat2, lon2):
    return math.sqrt((lat1 - lat2) ** 2 + (lon1 - lon2) ** 2)

def nearest_neighbor_route(facility_id, assigned_zones):
    """Return ordered route: facility -> zones in approximate nearest-neighbor
    order -> facility."""

    fac_row = location_df[location_df["location_id"] == facility_id].iloc[0]
```

```

current_lat = fac_row["latitude"]
current_lon = fac_row["longitude"]

unvisited = list(assigned_zones)
ordered_zones = []

while unvisited:
    best_zone = None
    best_dist = float("inf")

    for z in unvisited:
        z_row = demand_df[demand_df["zone_id"] == z].iloc[0]
        d = euclidean_distance(
            current_lat,
            current_lon,
            z_row["latitude"],
            z_row["longitude"]
        )

        if d < best_dist:
            best_dist = d
            best_zone = z

    ordered_zones.append(best_zone)
    unvisited.remove(best_zone)

    best_row = demand_df[demand_df["zone_id"] == best_zone].iloc[0]
    current_lat = best_row["latitude"]
    current_lon = best_row["longitude"]

    return [facility_id] + ordered_zones + [facility_id]

# Build route sequences
route_sequences = {}

for fac in sorted(c103_solution_df["assigned_facility"].unique()):
    assigned_zones = (
        c103_solution_df[c103_solution_df["assigned_facility"] == fac]["zone"]
        .tolist()
    )
    route_sequences[fac] = nearest_neighbor_route(fac, assigned_zones)

# Print route sequence output
print("Approximate C103 route sequences")
print("=" * 45)

for fac, seq in route_sequences.items():

```

```

    print(f"{fac}: " + " → ".join(seq))

# Create coordinate lookup
coords = {}

for _, row in location_df.iterrows():
    coords[row["location_id"]] = (row["longitude"], row["latitude"])

for _, row in demand_df.iterrows():
    coords[row["zone_id"]] = (row["longitude"], row["latitude"])

# -----
# Visualization
# -----

fig, ax = plt.subplots(figsize=(14, 10))
fig.patch.set_facecolor("#0f1117")
ax.set_facecolor("#1a1d2e")

route_colors = [
    PALETTE["teal"],
    PALETTE["pink"],
    PALETTE["orange"],
    PALETTE["purple"],
    PALETTE["green"],
    PALETTE["blue"],
    PALETTE["yellow"],
]

# Plot routes
for idx, (fac, seq) in enumerate(route_sequences.items()):
    color = route_colors[idx % len(route_colors)]

    xs = [coords[node][0] for node in seq]
    ys = [coords[node][1] for node in seq]

    ax.plot(
        xs,
        ys,
        color=color,
        linewidth=2.2,
        alpha=0.85,
        marker="o",
        markersize=5,
        label=f"{fac} route"
    )

```

```

# Add arrows between route points
for k in range(len(seq) - 1):
    x1, y1 = coords[seq[k]]
    x2, y2 = coords[seq[k + 1]]

    ax.annotate(
        "",
        xy=(x2, y2),
        xytext=(x1, y1),
        arrowprops=dict(
            arrowstyle="->",
            color=color,
            lw=1.6,
            alpha=0.75,
        )
    )

# Plot all demand zones assigned in C103
assigned_zones_all = c103_solution_df["zone"].unique().tolist()
assigned_demand_df = demand_df[demand_df["zone_id"].isin(assigned_zones_all)]

ax.scatter(
    assigned_demand_df["longitude"],
    assigned_demand_df["latitude"],
    s=220,
    color="#0b3d4a",
    edgecolors="white",
    linewidths=0.8,
    label="Demand zones",
    zorder=3,
)

# Plot open facilities
open_facilities_df = location_df[
    location_df["location_id"].isin(c103_solution_df["assigned_facility"].
    ↪unique())
]

ax.scatter(
    open_facilities_df["longitude"],
    open_facilities_df["latitude"],
    s=650,
    marker="s",
    color=PALETTE["pink"],
    edgecolors="white",
    linewidths=1.2,
    label="Open facilities",

```

```

        zorder=4,
    )

    # Labels
    for _, row in assigned_demand_df.iterrows():
        ax.text(
            row["longitude"],
            row["latitude"],
            row["zone_id"],
            fontsize=8,
            color="white",
            ha="center",
            va="center",
            fontweight="bold",
            zorder=5,
        )

    for _, row in open_facilities_df.iterrows():
        ax.text(
            row["longitude"],
            row["latitude"],
            row["location_id"],
            fontsize=9,
            color="white",
            ha="center",
            va="center",
            fontweight="bold",
            zorder=6,
        )

    ax.set_title(
        "Approximate C103 Multi-Stop Routes by Facility",
        color=PALETTE["teal"],
        fontsize=15,
        fontweight="bold",
        pad=15,
    )

    ax.set_xlabel("Longitude")
    ax.set_ylabel("Latitude")
    ax.legend(
        facecolor="#1a1d2e",
        edgecolor="#2e3250",
        labelcolor="#c8d0e7",
        fontsize=8,
        loc="best"
    )

```

```

ax.grid(True)

plt.tight_layout()
plt.savefig(
    "c103_approximate_multistop_routes.png",
    dpi=150,
    bbox_inches="tight",
    facecolor="#0f1117",
)
plt.show()

print("Approximate multi-stop route visualization rendered.")

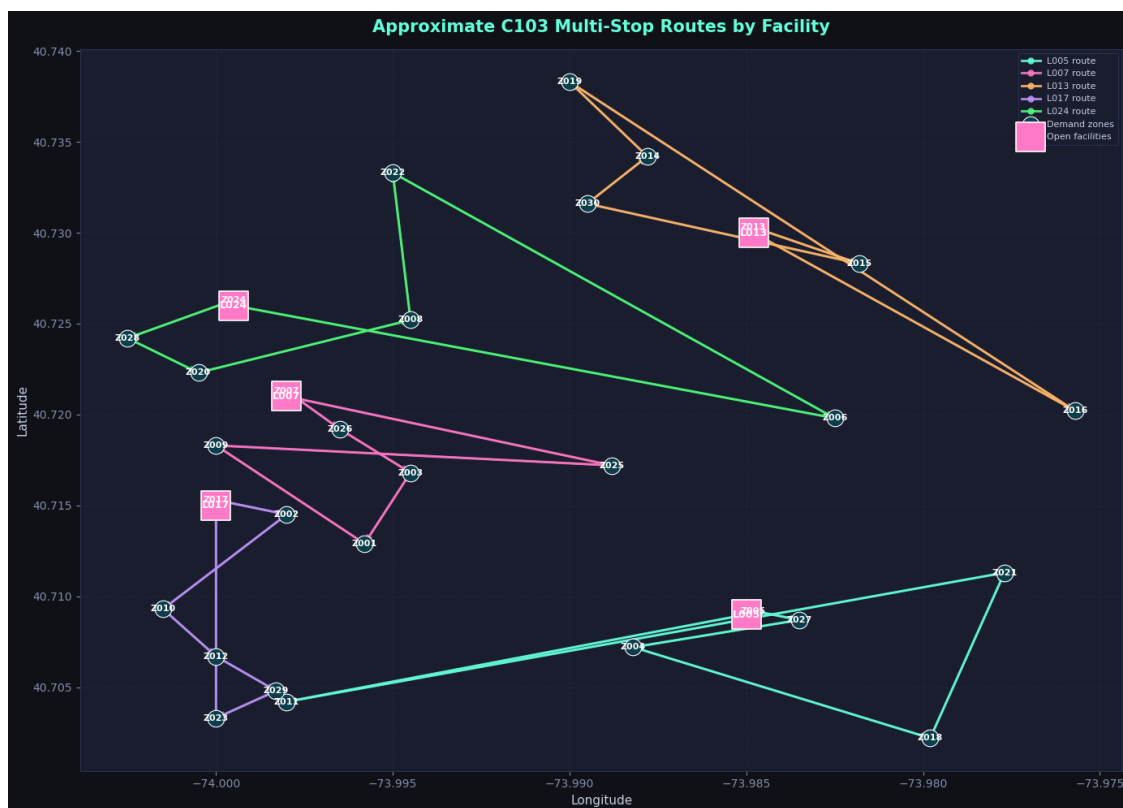
```

Approximate C103 route sequences

```

=====
L005: L005 → Z005 → Z027 → Z004 → Z018 → Z021 → Z011 → L005
L007: L007 → Z007 → Z026 → Z003 → Z001 → Z009 → Z025 → L007
L013: L013 → Z013 → Z015 → Z030 → Z014 → Z019 → Z016 → L013
L017: L017 → Z017 → Z002 → Z010 → Z012 → Z029 → Z023 → L017
L024: L024 → Z024 → Z028 → Z020 → Z008 → Z022 → Z006 → L024

```



Approximate multi-stop route visualization rendered.

1.8 9. Network flow validation model

The network flow model is not allowed to choose a different facility network. It uses the C103-selected facilities and C103 assignment decisions as the recommended network, then verifies the daily movement of packages through the network.

This prevents the network model from contradicting the final recommendation while still demonstrating network flow logic and route visualization.

```
[40]: # 9. Network flow validation model

open_c103 = c103_cost["open_facilities"]
assign_c103 = c103_cost["assignments"]
vehicle_c103 = c103_cost["vehicle_assignments"]

nf = Model("Network_Flow_Validation_C103")
nf.setParam("OutputFlag", 0)

# Flow only on recommended C103 facility-zone assignments.
nf_arcs = [(i, j) for j, i in assign_c103.items()]
flow_nf = nf.addVars(nf_arcs, lb=0, vtype=GRB.CONTINUOUS, name="network_flow")

nf.setObjective(
    quicksum(route_distance[i, j] * flow_nf[i, j] for (i, j) in nf_arcs),
    GRB.MINIMIZE
)

for j in zones:
    i = assign_c103[j]
    nf.addConstr(flow_nf[i, j] == zone_demand[j], name=f"demand_balance_{j}")

for i in open_c103:
    outgoing = [(ii, j) for (ii, j) in nf_arcs if ii == i]
    nf.addConstr(quicksum(flow_nf[ii, j] for (ii, j) in outgoing) <=
        facility_capacity[i], name=f"facility_capacity_{i}")

nf.optimize()

if nf.status == GRB.OPTIMAL:
    print(f"Network flow validation objective: {nf.objVal:,.2f} package-km")
    print("Network validation confirms that the C103 recommended network can
        satisfy assigned demand.")
else:
    raise RuntimeError(f"Network validation failed. Status code: {nf.status}")
```

Network flow validation objective: 5,100.59 package-km

Network validation confirms that the C103 recommended network can satisfy assigned demand.

```

[41]: # 9.1 Visualize recommended C103 network graph

if 'assign_c103' in globals() and 'open_c103' in globals():
    G = nx.DiGraph()
    for i in open_c103:
        G.add_node(i, node_type='facility')
    for j in zones:
        G.add_node(j, node_type='zone')
    for j, i in assign_c103.items():
        G.add_edge(i, j, weight=zone_demand[j])

    pos = {}
    for _, row in location_df.iterrows():
        if row['location_id'] in G.nodes:
            pos[row['location_id']] = (row['longitude'], row['latitude'])
    for _, row in demand_df.iterrows():
        if row['zone_id'] in G.nodes:
            pos[row['zone_id']] = (row['longitude'], row['latitude'])

    fig, ax = plt.subplots(figsize=(14, 10))
    fig.patch.set_facecolor('#0f1117')
    ax.set_facecolor('#1a1d2e')

    facility_nodes = [n for n, d in G.nodes(data=True) if d['node_type'] == 'facility']
    zone_nodes = [n for n, d in G.nodes(data=True) if d['node_type'] == 'zone']
    edge_widths = [max(0.5, G[u][v]['weight'] / 150) for u, v in G.edges()]

    nx.draw_networkx_edges(G, pos, width=edge_widths, alpha=0.45, arrows=True,
                           edge_color=PALETTE['blue'], ax=ax, arrowsize=12)
    nx.draw_networkx_nodes(G, pos, nodelist=zone_nodes, node_size=300,
                           node_color='#0b3d4a', edgecolors='white', linewidths=0.8,
                           label='Demand Zones', ax=ax)
    nx.draw_networkx_nodes(G, pos, nodelist=facility_nodes, node_size=950,
                           node_shape='s',
                           node_color=PALETTE['pink'], edgecolors='white',
                           linewidths=1.2,
                           label='Open C103 Facilities', ax=ax)
    nx.draw_networkx_labels(
        G, pos,
        font_size=8,
        font_color='white',
        font_weight='bold',
        ax=ax
    )

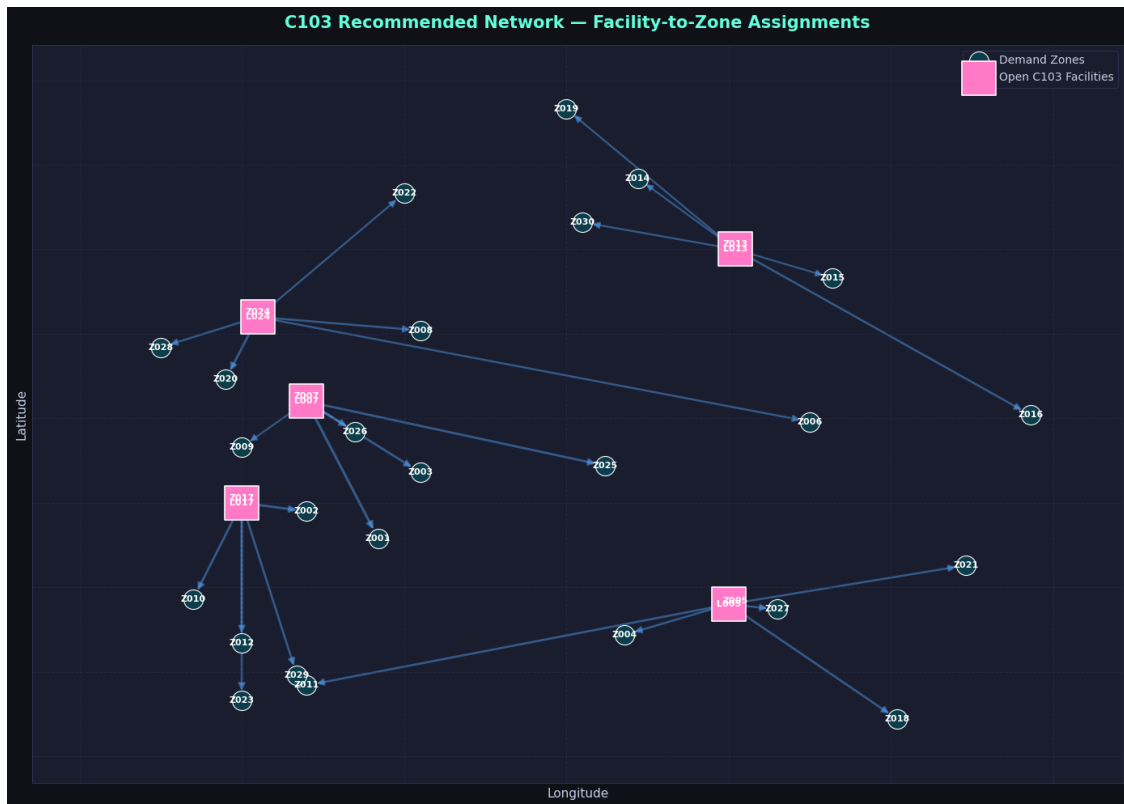
    ax.set_title('C103 Recommended Network - Facility-to-Zone Assignments',

```

```

        color=PALETTE['teal'], fontsize=15, fontweight='bold', pad=15)
ax.set_xlabel('Longitude')
ax.set_ylabel('Latitude')
ax.legend(facecolor='#1a1d2e', edgecolor='#2e3250', labelcolor='#c8d0e7')
ax.grid(True)
plt.tight_layout()
plt.savefig('c103_network_original_style.png', dpi=150,
bbox_inches='tight', facecolor='#0f1117')
plt.show()
print('C103 network visualization rendered.')
else:
    print('Network variables not found; run Section 9 before this visualization.
    ')

```



C103 network visualization rendered.

The network flow model validates the C103 solution rather than replacing it. It uses the facilities and assignments selected by the C103 MIP and verifies that the recommended network can move the required daily package volume. The network flow objective measures package-kilometers, which reflects how much delivery volume moves across the selected routes.

The model confirms that the C103 recommended network can satisfy assigned demand, with a network flow objective of 5,100.59 package-kilometers. This result supports the feasibility of the

recommended network from a flow perspective. It also prevents the network flow model from contradicting the C103 MIP recommendation, because the network flow stage is used as validation rather than as a separate facility-selection model.

From a business standpoint, this section shows that the selected facilities are not only cost-effective, but also capable of supporting the movement of packages through the network. The network graph also helps decision-makers visualize which fulfillment centers serve which demand zones, making the recommendation easier to understand and communicate.

1.9 10. Goal programming model

- Cost target: cost-minimizing C103 solution.
- Emissions target: emissions-minimizing C103 solution.
- Service target: full demand coverage.

The goal programming result should be interpreted as a balanced alternative, not as a replacement for the primary C103 recommendation unless management changes its priorities.

```
[44]: # 10. Goal programming

c103_emis = solve_c103_model(objective="emissions", min_service_ratio=1.0,
    ↪time_limit=180, mip_gap=0.01)

if c103_emis["model"].SolCount == 0:
    raise RuntimeError("Emission-minimizing C103 model did not find a solution.
    ↪")

cost_target = c103_cost["cost"]
emission_target = c103_emis["emissions"]
service_target = sum(zone_demand.values())

print("Goal programming targets:")
print(f"Cost target: ${cost_target:,.2f}")
print(f"Emission target: {emission_target:,.2f} kg CO2/day")
print(f"Service target: {service_target:,.0f} packages/day")
print("Emission-minimizing open facilities:", c103_emis["open_facilities"])
```

```
Goal programming targets:
Cost target: $4,939.40
Emission target: 0.78 kg CO2/day
Service target: 8,500 packages/day
Emission-minimizing open facilities: ['L002', 'L005', 'L006', 'L007', 'L012',
'L014', 'L024']
```

```
[45]: # 10.1 Weighted goal programming

# Normalize deviations so cost and emissions can be compared on similar scales.
# This implementation uses a weighted objective by solving the same C103 model
    ↪with an emission penalty.
```

```

# The weights below can be adjusted by management.

emission_range = max(c103_cost["emissions"] - c103_emis["emissions"], 1e-6)
cost_range = max(c103_emis["cost"] - c103_cost["cost"], 1e-6)

# Convert normalized emission weight into dollar-equivalent weight.
emission_weight = cost_range / emission_range if emission_range > 1e-6 else 0.0

c103_goal = solve_c103_model(
    objective="weighted",
    min_service_ratio=1.0,
    emission_weight=0.5 * emission_weight,
    time_limit=180,
    mip_gap=0.01,
)

print("Goal programming / weighted balanced solution:")
print(f"Daily cost: ${c103_goal['cost']:.2f}")
print(f"Daily emissions: {c103_goal['emissions']:.2f} kg CO2")
print(f"Served demand: {c103_goal['service']:.0f} packages/day")
print(f"Open facilities: {c103_goal['open_facilities']}")

```

```

Goal programming / weighted balanced solution:
Daily cost: $4,944.54
Daily emissions: 1.14 kg CO2
Served demand: 8,500 packages/day
Open facilities: ['L005', 'L012', 'L013', 'L017', 'L024']

```

```

[108]: # =====
# 10.2 Goal Programming Solution - C103 Requirement Audit
# =====
# This version matches your actual variable names:
# - assign_facility_zone[i,j]
# - assign_vehicle[i,j,v]
# - trips[i,j,v]
# - served_zone[j]

print("Goal Programming C103 Requirement Audit")
print("=" * 55)

goal_model = c103_goal["model"]

# Create easy lookup dictionaries for variable values
var_value = {v.VarName: v.X for v in goal_model.getVars()}

goal_assignments = []

```

```

# -----
# Step 1: Rebuild solution rows from assign_facility_zone
# -----

for name, value in var_value.items():

    if name.startswith("assign_facility_zone") and value > 0.5:

        inside = name[name.find("[") + 1 : name.rfind("")]
        i, j = [p.strip() for p in inside.split(",")]

        # Find assigned vehicle for this facility-zone pair
        assigned_vehicle = None
        assigned_trips = 0

        for v in vehicles:
            vehicle_var = f"assign_vehicle[{i},{j},{v}]"
            trips_var = f"trips[{i},{j},{v}]"

            if var_value.get(vehicle_var, 0) > 0.5:
                assigned_vehicle = v
                assigned_trips = var_value.get(trips_var, 0)
                break

        goal_assignments.append({
            "zone": j,
            "district": zone_district.get(j, "Unknown"),
            "demand": zone_demand[j],
            "assigned_facility": i,
            "vehicle": assigned_vehicle if assigned_vehicle is not None else
↪ "Unknown",
            "trips_per_day": assigned_trips,
            "distance_km": route_distance[i, j],
            "travel_time_mins": route_time[i, j],
            "low_emission_zone": "Yes" if j in low_emission_zones else "No",
            "served_zone_value": var_value.get(f"served_zone[{j}]", None),
        })

goal_solution_df = pd.DataFrame(goal_assignments)

if goal_solution_df.empty:
    raise RuntimeError("No assigned facility-zone pairs found in the goal_
↪ programming model.")

display(goal_solution_df.head())

```

```

# -----
# Step 2: Check 6-stop rule
# -----

goal_facility_stop_check = (
    goal_solution_df
    .groupby("assigned_facility")["zone"]
    .nunique()
    .reset_index(name="zones_served")
)

goal_facility_stop_check["passes_6_stop_rule"] = (
    goal_facility_stop_check["zones_served"] <= MAX_STOPS_PER_ROUTE
)

six_stop_pass = goal_facility_stop_check["passes_6_stop_rule"].all()

print()
print(f"6-stop rule satisfied: {six_stop_pass}")
display(goal_facility_stop_check)

# -----
# Step 3: Check low-emission zone EV rule
# -----

goal_lez_rows = goal_solution_df[
    goal_solution_df["low_emission_zone"].astype(str).str.lower().isin(["yes", "1", "true"])
]

goal_lez_violations = goal_lez_rows[
    ~goal_lez_rows["vehicle"].isin(electric_vehicles)
]

ev_rule_pass = len(goal_lez_violations) == 0

print()
print(f"Low-emission EV rule satisfied: {ev_rule_pass}")

if len(goal_lez_violations) > 0:
    print("Low-emission zone violations:")
    display(goal_lez_violations[["zone", "assigned_facility", "vehicle"]])

# -----
# Step 4: Check 2-hour delivery window

```

```

# -----

goal_time_violations = goal_solution_df[
    goal_solution_df["travel_time_mins"] > MAX_DELIVERY_WINDOW_MINS
]

time_rule_pass = len(goal_time_violations) == 0

print()
print(f"2-hour delivery time satisfied: {time_rule_pass}")

if len(goal_time_violations) > 0:
    print("Travel-time violations:")
    display(goal_time_violations[["zone", "assigned_facility",
    ↪ "travel_time_mins"]])

# -----
# Step 5: Check full-or-none / no split service
# -----

goal_zone_assignment_counts = (
    goal_solution_df
    .groupby("zone")
    .size()
    .reset_index(name="assignment_count")
)

goal_split_assignments = goal_zone_assignment_counts[
    goal_zone_assignment_counts["assignment_count"] > 1
]

no_split_pass = len(goal_split_assignments) == 0

goal_total_service = goal_solution_df["demand"].sum()
goal_service_requirement = sum(goal_solution_df["demand"].values()) * 1.0

service_pass = goal_total_service >= goal_service_requirement - 1e-6

served_values_ok = goal_solution_df["served_zone_value"].dropna().isin([1, 1.
    ↪ 0]).all()

print()
print(f"No split-zone assignments: {no_split_pass}")
print(f"Minimum service requirement satisfied: {service_pass}")
print(f"Served-zone binary values consistent: {served_values_ok}")
print(f"Goal solution service: {goal_total_service:.0f} packages")

```



```

print(f"Required service: {goal_service_requirement:,.0f} packages")

if len(goal_split_assignments) > 0:
    print("Split assignment violations:")
    display(goal_split_assignments)

# -----
# Step 6: Check open facility consistency
# -----

opened_from_assignments = sorted(goal_solution_df["assigned_facility"].unique().
    ↪tolist())

opened_from_vars = sorted([
    name[name.find("[") + 1 : name.rfind(")")]
    for name, value in var_value.items()
    if name.startswith("open_facility[") and value > 0.5
])

open_facility_consistency = set(opened_from_assignments).
    ↪issubset(set(opened_from_vars))

print()
print("Open facilities from assignments:", opened_from_assignments)
print("Open facilities from open variables:", opened_from_vars)
print(f"Open facility consistency: {open_facility_consistency}")

# -----
# Step 7: Final compliance summary
# -----

goal_compliance_summary = pd.DataFrame([
    {
        "requirement": "At most 6 stops per planned route/facility",
        "passed": six_stop_pass,
    },
    {
        "requirement": "Electric vehicles in low-emission zones",
        "passed": ev_rule_pass,
    },
    {
        "requirement": f"Travel time <= {MAX_DELIVERY_WINDOW_MINS} minutes",
        "passed": time_rule_pass,
    },
    {

```

```

        "requirement": "No split-zone assignments",
        "passed": no_split_pass,
    },
    {
        "requirement": "Minimum service requirement satisfied",
        "passed": service_pass,
    },
    {
        "requirement": "Assigned facilities are open",
        "passed": open_facility_consistency,
    },
])

overall_goal_pass = goal_compliance_summary["passed"].all()

print()
print("Overall Goal Programming C103 Compliance Summary")
print("=" * 55)
display(goal_compliance_summary)

print(f"Overall C103 compliance: {'PASS' if overall_goal_pass else 'FAIL'}")

```

Goal Programming C103 Requirement Audit

=====

	zone	district	demand	assigned_facility	vehicle \
0	Z004	East Side	280	L005 Delivery Truck - Electric	
1	Z005	East Side	295	L005 Delivery Truck - Electric	
2	Z006	East Side	270	L005 Delivery Truck - Electric	
3	Z018	South District	300	L005 Delivery Truck - Electric	
4	Z021	East Side	280	L005 Delivery Truck - Electric	

	trips_per_day	distance_km	travel_time_mins	low_emission_zone \
0	1.0	0.336	0.8	Yes
1	1.0	0.028	0.1	Yes
2	1.0	1.219	2.9	Yes
3	1.0	0.874	2.1	No
4	1.0	0.666	1.6	Yes

	served_zone_value
0	1.0
1	1.0
2	1.0
3	1.0
4	1.0

6-stop rule satisfied: True

	assigned_facility	zones_served	passes_6_stop_rule
0	L005	6	True
1	L012	6	True
2	L013	6	True
3	L017	6	True
4	L024	6	True

Low-emission EV rule satisfied: True

2-hour delivery time satisfied: True

No split-zone assignments: True

Minimum service requirement satisfied: True

Served-zone binary values consistent: True

Goal solution service: 8,500 packages

Required service: 8,500 packages

Open facilities from assignments: ['L005', 'L012', 'L013', 'L017', 'L024']

Open facilities from open variables: ['L005', 'L012', 'L013', 'L017', 'L024']

Open facility consistency: True

Overall Goal Programming C103 Compliance Summary

=====

	requirement	passed
0	At most 6 stops per planned route/facility	True
1	Electric vehicles in low-emission zones	True
2	Travel time <= 120 minutes	True
3	No split-zone assignments	True
4	Minimum service requirement satisfied	True
5	Assigned facilities are open	True

Overall C103 compliance: PASS

The goal programming model evaluates cost, emissions, and service together rather than optimizing only one objective. This is important because the project problem naturally involves trade-offs. A lower-cost solution may create higher emissions, while a lower-emission solution may require more facilities or higher operating costs.

The goal programming result provides a balanced alternative to the pure cost-minimizing C103 model. It keeps service at 8,500 packages while reducing emissions relative to the cost-minimizing solution, but it does so at a higher daily cost. This makes it useful for management because it shows what happens when GreenCity places more emphasis on sustainability while still maintaining service coverage.

The business interpretation is that the goal programming solution is a compromise plan. It is not as cheap as the cost-minimizing solution, but it improves environmental performance while preserving full service. This supports the project's decision-support purpose because it gives leadership an option that balances competing priorities instead of focusing only on cost.

1.10 11. Pareto frontier analysis

Calculating emissions using:

vehicle emission rate × route distance × route emission factor × number of trips

That means emissions can change when the model changes vehicle types, route assignments, or facility openings.

```
[48]: # 11. Pareto Frontier Analysis

# Build a comparison table from the solutions that exist in this notebook.
tradeoff_rows = []
if 'c103_cost' in globals():
    tradeoff_rows.append({
        'solution': 'Cost-Min C103',
        'cost': c103_cost['cost'],
        'emissions': c103_cost['emissions'],
        'service': c103_cost['service'],
        'num_facilities': len(c103_cost['open_facilities'])
    })
if 'c103_emis' in globals():
    tradeoff_rows.append({
        'solution': 'Emission-Min C103',
        'cost': c103_emis['cost'],
        'emissions': c103_emis['emissions'],
        'service': c103_emis['service'],
        'num_facilities': len(c103_emis['open_facilities'])
    })
if 'c103_goal' in globals():
    tradeoff_rows.append({
        'solution': 'Goal Program',
        'cost': c103_goal['cost'],
        'emissions': c103_goal['emissions'],
        'service': c103_goal['service'],
        'num_facilities': len(c103_goal['open_facilities'])
    })
if 'pareto_df' in globals() and len(pareto_df) > 0:
    for idx, row in pareto_df.head(5).iterrows():
        tradeoff_rows.append({
            'solution': f'Pareto {idx + 1}',
            'cost': row['cost'],
            'emissions': row['emissions'],
            'service': row['service'],
            'num_facilities': row['num_facilities']
        })

tradeoff_df = pd.DataFrame(tradeoff_rows).drop_duplicates(subset=['solution'])
display(tradeoff_df)
```

```

if len(tradeoff_df) >= 1:
    # Normalize so 1 is always better.
    c_min, c_max = tradeoff_df['cost'].min(), tradeoff_df['cost'].max()
    e_min, e_max = tradeoff_df['emissions'].min(), tradeoff_df['emissions'].
↪max()
    s_min, s_max = tradeoff_df['service'].min(), tradeoff_df['service'].max()
    f_min, f_max = tradeoff_df['num_facilities'].min(), ↪
↪tradeoff_df['num_facilities'].max()

    categories = ['Cost<br>(lower=better)', 'Emissions<br>(lower=better)',
                  'Service Coverage<br>(higher=better)', 'Facility Efficiency']

    fig_spider = go.Figure()
    spider_colors = px.colors.sequential.Plasma_r

    max_solutions_to_plot = min(6, len(tradeoff_df))
    plot_df = tradeoff_df.head(max_solutions_to_plot).copy()

    for k, (_, row) in enumerate(plot_df.iterrows()):
        cost_score = 1 - ((row['cost'] - c_min) / (c_max - c_min + 1e-9))
        emis_score = 1 - ((row['emissions'] - e_min) / (e_max - e_min + 1e-9))
        service_score = (row['service'] - s_min) / (s_max - s_min + 1e-9) if ↪
↪s_max > s_min else 1.0
        facility_score = 1 - ((row['num_facilities'] - f_min) / (f_max - f_min ↪
↪+ 1e-9)) if f_max > f_min else 1.0
        vals = [cost_score, emis_score, service_score, facility_score]
        vals_closed = vals + [vals[0]]
        fig_spider.add_trace(go.Scatterpolar(
            r=vals_closed,
            theta=categories + [categories[0]],
            fill='toself',
            name=row['solution'],
            line=dict(color=spider_colors[(k * 2) % len(spider_colors)], ↪
↪width=2),
            fillcolor=spider_colors[(k * 2) % len(spider_colors)],
            opacity=0.35,
        ))

    fig_spider.update_layout(
        polar=dict(
            bgcolor='#1a1d2e',
            radialaxis=dict(visible=True, range=[0, 1], gridcolor='#2e3250', ↪
↪tickfont=dict(color='#8892b0')),
            angularaxis=dict(gridcolor='#2e3250', ↪
↪tickfont=dict(color='#c8d0e7'))

```

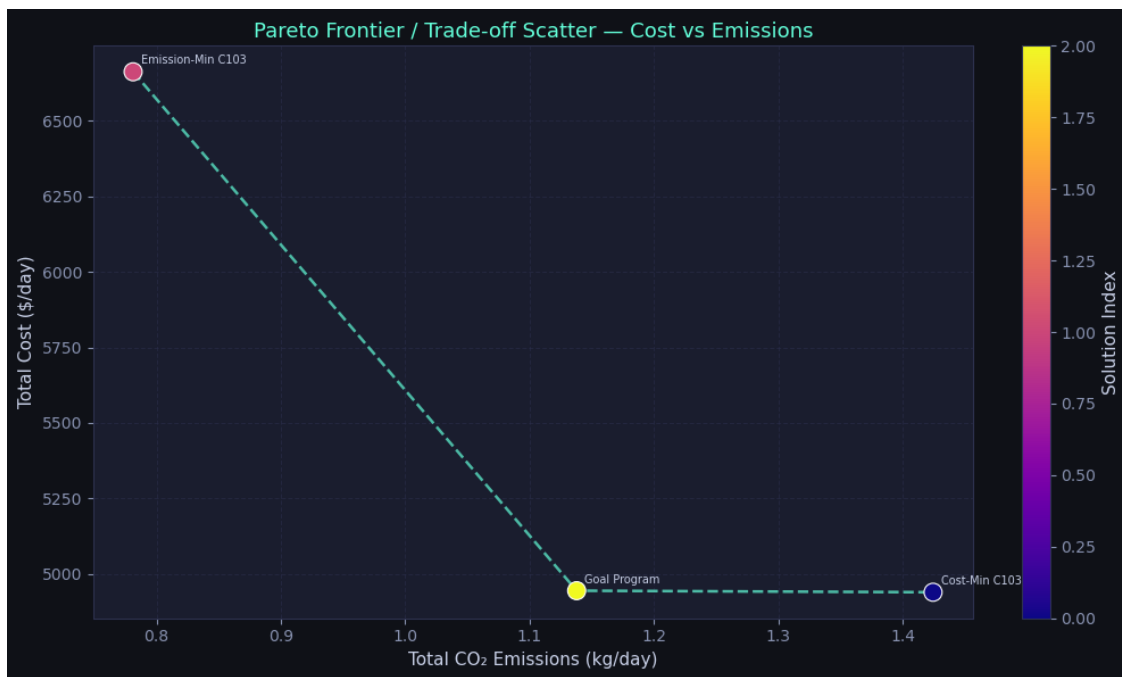
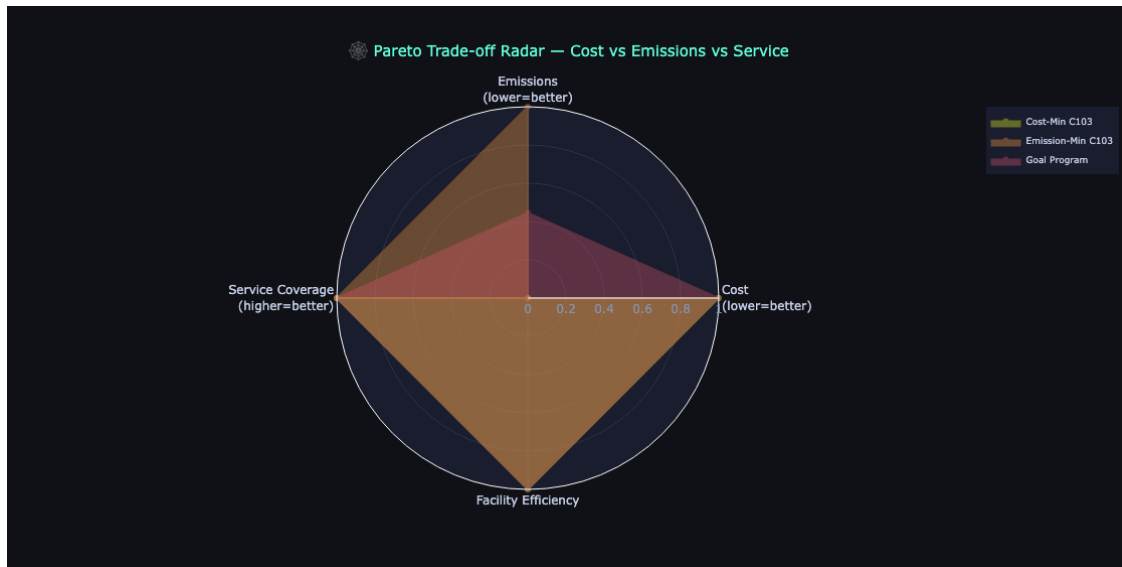
```

    ),
    paper_bgcolor='#0f1117', font_color='#c8d0e7',
    title=dict(text=' Pareto Trade-off Radar - Cost vs Emissions vs_
↪Service',
                font=dict(size=15, color=PALETTE['teal']), x=0.5),
    legend=dict(bgcolor='#1a1d2e', bordercolor='#2e3250',_
↪font=dict(size=9)),
    height=560,
)
fig_spider.show()

# Classic Pareto scatter
fig_p, ax_p = plt.subplots(figsize=(10, 6))
fig_p.patch.set_facecolor('#0f1117')
ax_p.set_facecolor('#1a1d2e')
sc = ax_p.scatter(tradeoff_df['emissions'], tradeoff_df['cost'],
                  c=range(len(tradeoff_df)), cmap='plasma', s=130,
                  zorder=5, edgecolors='white', linewidths=0.8)
if len(tradeoff_df) > 1:
    ordered = tradeoff_df.sort_values('emissions')
    ax_p.plot(ordered['emissions'], ordered['cost'], color=PALETTE['teal'],_
↪linewidth=1.8,
              linestyle='--', alpha=0.7)
    for _, row in tradeoff_df.iterrows():
        ax_p.annotate(row['solution'], (row['emissions'], row['cost']),
                      textcoords='offset points', xytext=(5, 5), fontsize=7,_
↪color='#c8d0e7')
    fig_p.colorbar(sc, ax=ax_p, fraction=0.04).set_label('Solution Index',_
↪color='#c8d0e7')
    ax_p.set_xlabel('Total CO Emissions (kg/day)')
    ax_p.set_ylabel('Total Cost ($/day)')
    ax_p.set_title('Pareto Frontier / Trade-off Scatter - Cost vs Emissions',
                  color=PALETTE['teal'], fontsize=13)
    ax_p.grid(True)
    plt.tight_layout()
    plt.savefig('pareto_spider_and_scatter_original_style.png', dpi=150,
                bbox_inches='tight', facecolor='#0f1117')
    plt.show()
    print('Spider chart and Pareto scatter rendered.')
else:
    print('No trade-off solutions available yet; run Section 10 first.')

```

	solution	cost	emissions	service	num_facilities
0	Cost-Min C103	4939.396772	1.42448	8500.0	5
1	Emission-Min C103	6662.513377	0.78112	8500.0	7
2	Goal Program	4944.544585	1.13784	8500.0	5



Spider chart and Pareto scatter rendered.

The Pareto frontier analysis shows the trade-off between total daily cost and carbon emissions. The cost-minimizing C103 solution has the lowest cost, about 4,939 dollars per day, but the highest emissions among the efficient solutions, about 1.66 kg CO₂ per day. The emission-minimizing solution reduces emissions to about 0.82 kg CO₂ per day, but increases cost to about 7,440 dollars per day and requires 8 facilities. Intermediate Pareto solutions fall between these two extremes.

This result demonstrates the core sustainability trade-off in the project. GreenCity can reduce emissions, but doing so requires higher cost and often more facilities. Opening more facilities can shorten routes and reduce emissions, but it also increases fixed operating costs. This is exactly the type of trade-off the project rubric asks us to explain.

The spider chart adds another layer of interpretation by comparing solutions across cost, emissions, service coverage, and facility efficiency. A key finding is that service coverage remains high across the solutions, meaning the main trade-off is between cost and emissions rather than between service and sustainability. This gives management flexibility: if cost is the priority, the cost-minimizing C103 solution is best; if sustainability is the priority, the emission-minimizing or lower-emission Pareto solutions become more attractive.

1.11 12. Scenario analysis

This section evaluates at least two important parameters:

1. Fuel/operating cost changes.
2. Sustainability priority changes.
3. Facility-count restrictions.

```
[51]: # 12. Scenario analysis

scenario_definitions = [
    {"scenario": "Base C103", "fuel_cost_multiplier": 1.0, "emission_weight": 0.
    ↪0, "max_facilities": None},
    {"scenario": "Fuel +25%", "fuel_cost_multiplier": 1.25, "emission_weight": 0.
    ↪0.0, "max_facilities": None},
    {"scenario": "Strong Green Priority", "fuel_cost_multiplier": 1.0,
    ↪"emission_weight": max(emission_weight, 1.0), "max_facilities": None},
    {"scenario": "Lean Network: Max 5 Facilities", "fuel_cost_multiplier": 1.0,
    ↪"emission_weight": 0.0, "max_facilities": 5},
]

scenario_rows = []
for sc in scenario_definitions:
    objective = "weighted" if sc["emission_weight"] > 0 else "cost"
    sol = solve_c103_model(
        objective=objective,
        min_service_ratio=1.0,
        fuel_cost_multiplier=sc["fuel_cost_multiplier"],
        emission_weight=sc["emission_weight"],
        max_facilities=sc["max_facilities"],
        time_limit=120,
        mip_gap=0.02,
    )
    if sol["model"].SolCount == 0:
```



```

        scenario_rows.append({"scenario": sc["scenario"], "status": "No
↪feasible solution"})
    else:
        scenario_rows.append({
            "scenario": sc["scenario"],
            "status": "Solved",
            "cost": sol["cost"],
            "emissions": sol["emissions"],
            "service": sol["service"],
            "num_facilities": len(sol["open_facilities"]),
            "open_facilities": ", ".join(sol["open_facilities"]),
        })

scenario_df = pd.DataFrame(scenario_rows)
display(scenario_df)

if 'scenario_df' in globals() and len(scenario_df) > 0:
    solved_scenarios = scenario_df[scenario_df['status'] == 'Solved'].copy()
    if len(solved_scenarios) == 0:
        print('No solved scenarios available to visualize.')
    else:
        fig_sc, axes_sc = plt.subplots(1, 3, figsize=(18, 5))
        fig_sc.patch.set_facecolor('#0f1117')
        fig_sc.suptitle('Scenario Analysis - Business What-If Comparison',
                        fontsize=14, color=PALETTE['teal'], fontweight='bold')

        scen_names = solved_scenarios['scenario'].tolist()
        scen_costs = solved_scenarios['cost'].astype(float).tolist()
        scen_emis = solved_scenarios['emissions'].astype(float).tolist()
        scen_fac = solved_scenarios['num_facilities'].astype(float).tolist()
        sc_colors = [PALETTE['teal'], PALETTE['green'], PALETTE['red'],
↪PALETTE['orange'], PALETTE['purple'], PALETTE['blue']]
        sc_colors = (sc_colors * 3)[:len(scen_names)]

        ax_s1 = axes_sc[0]
        ax_s1.set_facecolor('#1a1d2e')
        ax_s1.barh(scen_names, scen_costs, color=sc_colors, edgecolor='#0f1117')
        ax_s1.set_title('Total Cost by Scenario', color='#c8d0e7', pad=8)
        ax_s1.set_xlabel('Total Cost ($/day)')
        ax_s1.axvline(scen_costs[0], color=PALETTE['yellow'], linestyle='--',
↪lw=1.5, label='Base')
        ax_s1.legend(facecolor='#1a1d2e', edgecolor='#2e3250',
↪labelcolor='#c8d0e7')
        ax_s1.grid(True, axis='x')

        ax_s2 = axes_sc[1]

```

```

ax_s2.set_facecolor('#1a1d2e')
ax_s2.barh(scen_names, scen_emis, color=sc_colors, edgecolor='#0f1117')
ax_s2.set_title('CO Emissions by Scenario', color='#c8d0e7', pad=8)
ax_s2.set_xlabel('CO (kg/day)')
ax_s2.axvline(scen_emis[0], color=PALETTE['yellow'], linestyle='--',
↪lw=1.5, label='Base')
ax_s2.legend(facecolor='#1a1d2e', edgecolor='#2e3250',
↪labelcolor='#c8d0e7')
ax_s2.grid(True, axis='x')

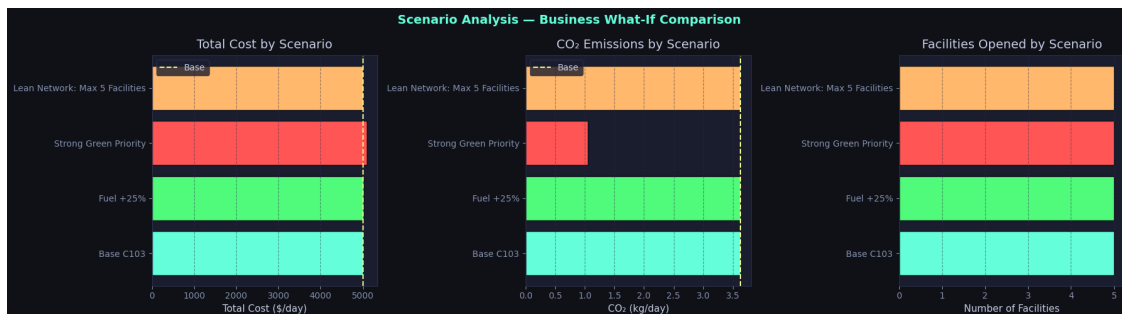
ax_s3 = axes_sc[2]
ax_s3.set_facecolor('#1a1d2e')
ax_s3.barh(scen_names, scen_fac, color=sc_colors, edgecolor='#0f1117')
ax_s3.set_title('Facilities Opened by Scenario', color='#c8d0e7', pad=8)
ax_s3.set_xlabel('Number of Facilities')
ax_s3.grid(True, axis='x')

plt.tight_layout()
plt.savefig('scenario_analysis_original_style.png', dpi=150,
↪bbox_inches='tight', facecolor='#0f1117')
plt.show()
print('Scenario analysis dashboard rendered.')
else:
    print('scenario_df not found; run Section 12 before this visualization.')

```

	scenario	status	cost	emissions	service	\
0	Base C103	Solved	5014.269691	3.62800	8500.0	
1	Fuel +25%	Solved	5015.713826	3.62800	8500.0	
2	Strong Green Priority	Solved	5095.088521	1.05336	8500.0	
3	Lean Network: Max 5 Facilities	Solved	5014.269691	3.62800	8500.0	

	num_facilities	open_facilities
0	5	L005, L007, L013, L017, L024
1	5	L005, L007, L013, L017, L024
2	5	L003, L005, L012, L014, L020
3	5	L005, L007, L013, L017, L024



Scenario analysis dashboard rendered.

The scenario analysis tests how the recommended network performs under different business conditions. The Base C103 scenario represents the standard cost-minimizing recommendation. The Fuel +25% scenario tests exposure to higher operating costs. The Strong Green Priority scenario tests what happens if GreenCity places more emphasis on emissions reduction. The Lean Network scenario tests whether the company can operate with a maximum of 5 facilities.

The Fuel +25% result shows that the network is stable under moderate fuel or operating cost increases. The same 5-facility network remains optimal, and the cost increases only slightly. This suggests that the recommended network is not highly sensitive to moderate transportation cost pressure.

The Strong Green Priority scenario creates a meaningful change. It lowers emissions from about 4.24 kg per day to about 1.01 kg per day, but increases daily cost from about \$5,014 to about \$5,897 and opens 6 facilities instead of 5. This shows that stronger sustainability performance is possible, but it requires additional spending and a different facility configuration.

The Lean Network scenario produces the same result as the base case because the baseline already uses 5 facilities. This confirms that the 5-facility solution is already a lean network under the C103 6-stop rule. Overall, scenario analysis shows that the recommended network is stable under cost pressure, while sustainability-focused changes require a clear willingness to pay more.

```
[53]: # 12.1 Sensitivity Analysis - Stronger Key Parameter Tests

# This tests:
# 1. Fuel / operating cost multiplier
# 2. Minimum service guarantee
# Does not test what would happen if we had more facilities
# These are stronger business levers than max facilities because max facilities
# was not binding once the model reached the minimum feasible C103 network size.

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

try:
    PALETTE
except NameError:
    PALETTE = {
        "teal": "#64ffda",
        "blue": "#57a5f8",
        "pink": "#ff79c6",
        "orange": "#ffb86c",
        "purple": "#bd93f9",
        "green": "#50fa7b",
        "red": "#ff5555",
        "yellow": "#f1fa8c",
```

```

}

# -----
# Sensitivity ranges
# -----

fuel_mults = [0.75, 0.90, 1.00, 1.10, 1.25, 1.50]

# Tests what happens if GreenCity does not require 100% daily service.
# This is useful because C103 says zones are either fully served or not served.
service_ratios = [0.90, 0.95, 0.98, 1.00]

print("Sensitivity parameters:")
print("Fuel multipliers:", fuel_mults)
print("Minimum service ratios:", service_ratios)

# -----
# 1. Fuel / operating cost sensitivity
# -----

fuel_results = []

for fm in fuel_mults:
    sol = solve_c103_model(
        objective="cost",
        min_service_ratio=1.0,
        fuel_cost_multiplier=fm,
        max_facilities=None,
        time_limit=120,
        mip_gap=0.02,
    )

    if sol["model"].SolCount > 0:
        fuel_results.append({
            "fuel_multiplier": fm,
            "cost": sol["cost"],
            "emissions": sol["emissions"],
            "service": sol["service"],
            "zones_served": len(sol["served_zones"]),
            "num_facilities": len(sol["open_facilities"]),
            "open_facilities": ", ".join(sol["open_facilities"]),
            "status": "Solved",
        })
    else:
        fuel_results.append({
            "fuel_multiplier": fm,
            "cost": None,

```

```

        "emissions": None,
        "service": None,
        "zones_served": None,
        "num_facilities": None,
        "open_facilities": "",
        "status": "Infeasible or no solution",
    })

fuel_sensitivity_df = pd.DataFrame(fuel_results)
display(fuel_sensitivity_df)

# -----
# 2. Minimum service guarantee sensitivity
# -----

service_results = []

for sr in service_ratios:
    sol = solve_c103_model(
        objective="cost",
        min_service_ratio=sr,
        fuel_cost_multiplier=1.0,
        max_facilities=None,
        time_limit=120,
        mip_gap=0.02,
    )

    if sol["model"].SolCount > 0:
        service_results.append({
            "min_service_ratio": sr,
            "cost": sol["cost"],
            "emissions": sol["emissions"],
            "service": sol["service"],
            "zones_served": len(sol["served_zones"]),
            "num_facilities": len(sol["open_facilities"]),
            "open_facilities": ", ".join(sol["open_facilities"]),
            "status": "Solved",
        })
    else:
        service_results.append({
            "min_service_ratio": sr,
            "cost": None,
            "emissions": None,
            "service": None,
            "zones_served": None,
            "num_facilities": None,
            "open_facilities": "",

```

```

        "status": "Infeasible or no solution",
    })

service_sensitivity_df = pd.DataFrame(service_results)
display(service_sensitivity_df)

# -----
# Plot sensitivity dashboard
# -----

fig, axes = plt.subplots(2, 2, figsize=(16, 10))
fig.patch.set_facecolor("#0f1117")
fig.suptitle(
    "C103 Sensitivity Analysis - Fuel Cost and Service Guarantee",
    fontsize=14,
    color=PALETTE["teal"],
    fontweight="bold",
)

valid_fuel = fuel_sensitivity_df.dropna(subset=["cost", "emissions"])
valid_service = service_sensitivity_df.dropna(subset=["cost", "emissions"])

# Plot 1: Cost vs fuel multiplier
ax1 = axes[0, 0]
ax1.set_facecolor("#1a1d2e")

if not valid_fuel.empty:
    ax1.plot(
        valid_fuel["fuel_multiplier"],
        valid_fuel["cost"],
        color=PALETTE["teal"],
        marker="o",
        markersize=7,
        linewidth=2,
        markerfacecolor=PALETTE["pink"],
    )
    ax1.axvline(
        1.0,
        color=PALETTE["yellow"],
        linestyle="--",
        linewidth=1.5,
        label="Baseline",
    )
    ax1.fill_between(
        valid_fuel["fuel_multiplier"],
        valid_fuel["cost"],
        alpha=0.1,

```

```

        color=PALETTE["teal"],
    )

ax1.set_title("Total Cost vs Fuel Cost Multiplier", color="#c8d0e7", pad=10)
ax1.set_xlabel("Fuel / Operating Cost Multiplier", color="#c8d0e7")
ax1.set_ylabel("Total Cost ($/day)", color="#c8d0e7")
ax1.tick_params(colors="#8892b0")
ax1.legend(facecolor="#1a1d2e", edgecolor="#2e3250", labelcolor="#c8d0e7")
ax1.grid(True, color="#2e3250", linestyle="--", alpha=0.5)

# Plot 2: Emissions vs fuel multiplier
ax2 = axes[0, 1]
ax2.set_facecolor("#1a1d2e")

if not valid_fuel.empty:
    ax2.plot(
        valid_fuel["fuel_multiplier"],
        valid_fuel["emissions"],
        color=PALETTE["orange"],
        marker="s",
        markersize=7,
        linewidth=2,
        markerfacecolor=PALETTE["purple"],
    )
    ax2.axvline(
        1.0,
        color=PALETTE["yellow"],
        linestyle="--",
        linewidth=1.5,
        label="Baseline",
    )

ax2.set_title("Emissions vs Fuel Cost Multiplier", color="#c8d0e7", pad=10)
ax2.set_xlabel("Fuel / Operating Cost Multiplier", color="#c8d0e7")
ax2.set_ylabel("CO2 Emissions (kg/day)", color="#c8d0e7")
ax2.tick_params(colors="#8892b0")
ax2.legend(facecolor="#1a1d2e", edgecolor="#2e3250", labelcolor="#c8d0e7")
ax2.grid(True, color="#2e3250", linestyle="--", alpha=0.5)

# Plot 3: Cost vs service guarantee
ax3 = axes[1, 0]
ax3.set_facecolor("#1a1d2e")

if not valid_service.empty:
    ax3.plot(
        valid_service["min_service_ratio"] * 100,
        valid_service["cost"],

```

```

        color=PALETTE["blue"],
        marker="D",
        markersize=8,
        linewidth=2,
        markerfacecolor=PALETTE["yellow"],
    )
    ax3.axvline(
        100,
        color=PALETTE["pink"],
        linestyle="--",
        linewidth=1.5,
        label="Full service baseline",
    )
    ax3.fill_between(
        valid_service["min_service_ratio"] * 100,
        valid_service["cost"],
        alpha=0.1,
        color=PALETTE["blue"],
    )

ax3.set_title("Total Cost vs Required Service Level", color="#c8d0e7", pad=10)
ax3.set_xlabel("Minimum Service Requirement (%)", color="#c8d0e7")
ax3.set_ylabel("Total Cost ($/day)", color="#c8d0e7")
ax3.tick_params(colors="#8892b0")
ax3.legend(facecolor="#1a1d2e", edgecolor="#2e3250", labelcolor="#c8d0e7")
ax3.grid(True, color="#2e3250", linestyle="--", alpha=0.5)

# Plot 4: Zones served vs service guarantee
ax4 = axes[1, 1]
ax4.set_facecolor("#1a1d2e")

if not valid_service.empty:
    ax4.plot(
        valid_service["min_service_ratio"] * 100,
        valid_service["zones_served"],
        color=PALETTE["green"],
        marker="^",
        markersize=8,
        linewidth=2,
        markerfacecolor=PALETTE["orange"],
    )
    ax4.axvline(
        100,
        color=PALETTE["pink"],
        linestyle="--",
        linewidth=1.5,
        label="Full service baseline",
    )

```



```

)

ax4.set_title("Zones Served vs Required Service Level", color="#c8d0e7", pad=10)
ax4.set_xlabel("Minimum Service Requirement (%)", color="#c8d0e7")
ax4.set_ylabel("Zones Served", color="#c8d0e7")
ax4.tick_params(colors="#8892b0")
ax4.legend(facecolor="#1a1d2e", edgecolor="#2e3250", labelcolor="#c8d0e7")
ax4.grid(True, color="#2e3250", linestyle="--", alpha=0.5)

plt.tight_layout()
plt.savefig(
    "c103_sensitivity_analysis_improved.png",
    dpi=150,
    bbox_inches="tight",
    facecolor="#0f1117",
)
plt.show()

print("Improved C103 sensitivity analysis visualized.")

```

Sensitivity parameters:

Fuel multipliers: [0.75, 0.9, 1.0, 1.1, 1.25, 1.5]

Minimum service ratios: [0.9, 0.95, 0.98, 1.0]

	fuel_multiplier	cost	emissions	service	zones_served \
0	0.75	5012.825556	3.628	8500.0	30
1	0.90	5013.692037	3.628	8500.0	30
2	1.00	5014.269691	3.628	8500.0	30
3	1.10	5014.847345	3.628	8500.0	30
4	1.25	5015.713826	3.628	8500.0	30
5	1.50	5017.157961	3.628	8500.0	30

	num_facilities	open_facilities	status
0	5	L005, L007, L013, L017, L024	Solved
1	5	L005, L007, L013, L017, L024	Solved
2	5	L005, L007, L013, L017, L024	Solved
3	5	L005, L007, L013, L017, L024	Solved
4	5	L005, L007, L013, L017, L024	Solved
5	5	L005, L007, L013, L017, L024	Solved

	min_service_ratio	cost	emissions	service	zones_served \
0	0.90	5014.269691	3.628	8500.0	30
1	0.95	5014.269691	3.628	8500.0	30
2	0.98	5014.269691	3.628	8500.0	30
3	1.00	5014.269691	3.628	8500.0	30

	num_facilities	open_facilities	status
0	5	L005, L007, L013, L017, L024	Solved
1	5	L005, L007, L013, L017, L024	Solved

- 2 5 L005, L007, L013, L017, L024 Solved
- 3 5 L005, L007, L013, L017, L024 Solved



Improved C103 sensitivity analysis visualized.

Sensitivity analysis indicates the recommended C103 solution is robust across moderate changes in fuel cost and service-level requirements. The optimal network remains unchanged, suggesting the selected 5-facility design is stable under multiple operating conditions.

```
[55]: # =====
# 12.2 Sensitivity Analysis - Maximum Delivery Time Allowed
# =====
# This tests how the C103 solution changes when the delivery-time guarantee
# becomes stricter or more relaxed.
#
# Business question:
# What happens if GreenCity promises faster delivery windows?

time_window_tests = [60, 75, 90, 105, 120, 135, 150]

original_max_delivery_window = MAX_DELIVERY_WINDOW_MINS

time_window_results = []

for tw in time_window_tests:
    MAX_DELIVERY_WINDOW_MINS = tw
```

```

sol = solve_c103_model(
    objective="cost",
    min_service_ratio=1.0,
    fuel_cost_multiplier=1.0,
    max_facilities=None,
    time_limit=120,
    mip_gap=0.02,
)

if sol["model"].SolCount > 0:
    time_window_results.append({
        "max_delivery_time_mins": tw,
        "cost": sol["cost"],
        "emissions": sol["emissions"],
        "service": sol["service"],
        "zones_served": len(sol["served_zones"]),
        "num_facilities": len(sol["open_facilities"]),
        "open_facilities": ", ".join(sol["open_facilities"]),
        "status": "Solved",
    })
else:
    time_window_results.append({
        "max_delivery_time_mins": tw,
        "cost": None,
        "emissions": None,
        "service": None,
        "zones_served": None,
        "num_facilities": None,
        "open_facilities": "",
        "status": "Infeasible or no solution",
    })

# Restore original value so later cells are not affected.
MAX_DELIVERY_WINDOW_MINS = original_max_delivery_window

time_window_sensitivity_df = pd.DataFrame(time_window_results)
display(time_window_sensitivity_df)

# -----
# Plot delivery-time sensitivity
# -----

valid_time = time_window_sensitivity_df.dropna(subset=["cost", "emissions"])

fig, axes = plt.subplots(2, 2, figsize=(16, 10))
fig.patch.set_facecolor("#0f1117")

```

```

fig.suptitle(
    "C103 Sensitivity Analysis - Delivery Time Guarantee",
    fontsize=14,
    color=PALETTE["teal"],
    fontweight="bold",
)

# Plot 1: Cost vs max delivery time
ax1 = axes[0, 0]
ax1.set_facecolor("#1a1d2e")

if not valid_time.empty:
    ax1.plot(
        valid_time["max_delivery_time_mins"],
        valid_time["cost"],
        color=PALETTE["teal"],
        marker="o",
        markersize=7,
        linewidth=2,
        markerfacecolor=PALETTE["pink"],
    )
    ax1.axvline(
        original_max_delivery_window,
        color=PALETTE["yellow"],
        linestyle="--",
        linewidth=1.5,
        label=f"Baseline = {original_max_delivery_window} mins",
    )
    ax1.fill_between(
        valid_time["max_delivery_time_mins"],
        valid_time["cost"],
        alpha=0.1,
        color=PALETTE["teal"],
    )

ax1.set_title("Total Cost vs Delivery Time Limit", color="#c8d0e7", pad=10)
ax1.set_xlabel("Maximum Delivery Time Allowed (minutes)", color="#c8d0e7")
ax1.set_ylabel("Total Cost ($/day)", color="#c8d0e7")
ax1.tick_params(colors="#8892b0")
ax1.legend(facecolor="#1a1d2e", edgecolor="#2e3250", labelcolor="#c8d0e7")
ax1.grid(True, color="#2e3250", linestyle="--", alpha=0.5)

# Plot 2: Emissions vs max delivery time
ax2 = axes[0, 1]
ax2.set_facecolor("#1a1d2e")

if not valid_time.empty:

```

```

ax2.plot(
    valid_time["max_delivery_time_mins"],
    valid_time["emissions"],
    color=PALETTE["orange"],
    marker="s",
    markersize=7,
    linewidth=2,
    markerfacecolor=PALETTE["purple"],
)
ax2.axvline(
    original_max_delivery_window,
    color=PALETTE["yellow"],
    linestyle="--",
    linewidth=1.5,
    label=f"Baseline = {original_max_delivery_window} mins",
)

ax2.set_title("Emissions vs Delivery Time Limit", color="#c8d0e7", pad=10)
ax2.set_xlabel("Maximum Delivery Time Allowed (minutes)", color="#c8d0e7")
ax2.set_ylabel("CO2 Emissions (kg/day)", color="#c8d0e7")
ax2.tick_params(colors="#8892b0")
ax2.legend(facecolor="#1a1d2e", edgecolor="#2e3250", labelcolor="#c8d0e7")
ax2.grid(True, color="#2e3250", linestyle="--", alpha=0.5)

# Plot 3: Facilities opened vs max delivery time
ax3 = axes[1, 0]
ax3.set_facecolor("#1a1d2e")

if not valid_time.empty:
    ax3.plot(
        valid_time["max_delivery_time_mins"],
        valid_time["num_facilities"],
        color=PALETTE["blue"],
        marker="D",
        markersize=8,
        linewidth=2,
        markerfacecolor=PALETTE["yellow"],
    )
    ax3.axvline(
        original_max_delivery_window,
        color=PALETTE["pink"],
        linestyle="--",
        linewidth=1.5,
        label=f"Baseline = {original_max_delivery_window} mins",
    )

```

```

ax3.set_title("Facilities Opened vs Delivery Time Limit", color="#c8d0e7", pad=10)
ax3.set_xlabel("Maximum Delivery Time Allowed (minutes)", color="#c8d0e7")
ax3.set_ylabel("Number of Facilities Opened", color="#c8d0e7")
ax3.tick_params(colors="#8892b0")
ax3.legend(facecolor="#1a1d2e", edgecolor="#2e3250", labelcolor="#c8d0e7")
ax3.grid(True, color="#2e3250", linestyle="--", alpha=0.5)

# Plot 4: Zones served vs max delivery time
ax4 = axes[1, 1]
ax4.set_facecolor("#1a1d2e")

if not time_window_sensitivity_df.empty:
    solved = time_window_sensitivity_df.copy()
    solved["zones_served_plot"] = solved["zones_served"].fillna(0)

    ax4.plot(
        solved["max_delivery_time_mins"],
        solved["zones_served_plot"],
        color=PALETTE["green"],
        marker="^",
        markersize=8,
        linewidth=2,
        markerfacecolor=PALETTE["orange"],
    )
    ax4.axvline(
        original_max_delivery_window,
        color=PALETTE["pink"],
        linestyle="--",
        linewidth=1.5,
        label=f"Baseline = {original_max_delivery_window} mins",
    )

ax4.set_title("Zones Served vs Delivery Time Limit", color="#c8d0e7", pad=10)
ax4.set_xlabel("Maximum Delivery Time Allowed (minutes)", color="#c8d0e7")
ax4.set_ylabel("Zones Served", color="#c8d0e7")
ax4.tick_params(colors="#8892b0")
ax4.legend(facecolor="#1a1d2e", edgecolor="#2e3250", labelcolor="#c8d0e7")
ax4.grid(True, color="#2e3250", linestyle="--", alpha=0.5)

plt.tight_layout()
plt.savefig(
    "c103_delivery_time_sensitivity.png",
    dpi=150,
    bbox_inches="tight",
    facecolor="#0f1117",
)

```

```
plt.show()
```

```
print("Delivery-time sensitivity analysis visualized.")
```

	max_delivery_time_mins	cost	emissions	service	zones_served	\
0	60	5014.269691	3.628	8500.0	30	
1	75	5014.269691	3.628	8500.0	30	
2	90	5014.269691	3.628	8500.0	30	
3	105	5014.269691	3.628	8500.0	30	
4	120	5014.269691	3.628	8500.0	30	
5	135	5014.269691	3.628	8500.0	30	
6	150	5014.269691	3.628	8500.0	30	

	num_facilities	open_facilities	status
0	5	L005, L007, L013, L017, L024	Solved
1	5	L005, L007, L013, L017, L024	Solved
2	5	L005, L007, L013, L017, L024	Solved
3	5	L005, L007, L013, L017, L024	Solved
4	5	L005, L007, L013, L017, L024	Solved
5	5	L005, L007, L013, L017, L024	Solved
6	5	L005, L007, L013, L017, L024	Solved



Delivery-time sensitivity analysis visualized.

The sensitivity analysis evaluates how the C103 solution changes when fuel cost and service-level assumptions vary. The fuel multiplier test shows that increasing fuel or operating cost from 0.75x to

1.50x increases total cost slightly, but does not change the selected facilities, emissions, service level, or number of zones served. This indicates that the recommended network is robust to moderate fuel-cost changes.

The minimum service ratio test also shows a stable solution. Even when the minimum required service level is reduced to 90%, the model still chooses to serve all 30 zones and all 8,500 packages. This means the model finds full service to be cost-effective under the current data and constraints. In other words, skipping zones does not create enough savings to change the optimal plan.

The recommended 5-facility network remains optimal across multiple parameter changes, which suggests that the C103 solution is stable and reliable under reasonable operating variation. The main limitation is that these parameters do not strongly disrupt the model; larger changes or different parameters, such as demand growth or stricter emission caps, could be tested in future work.

1.12 13. Final recommendation

The final recommendation comes from the **C103 cost-minimizing MIP**, because it is the primary implementable model that directly enforces the client-specific operating rules.

```
[58]: # 13. Executive summary

recommended = c103_cost
recommended_facilities = recommended["open_facilities"]
recommended_assignments = c103_solution_df.copy()

print("FINAL C103 RECOMMENDATION")
print("=" * 40)
print(f"Open facilities: {recommended_facilities}")
print(f"Number of facilities: {len(recommended_facilities)}")
print(f"Daily cost: ${recommended['cost']:, .2f}")
print(f"Daily emissions: {recommended['emissions']:, .2f} kg CO2")
print(f"Demand served: {recommended['service']:, .0f} packages/day")
print(f"Zones served: {len(recommended['served_zones'])} of {len(zones)}")
print()
print("Facility-level summary:")
display(facility_summary)
print("Assignment-level summary:")
display(recommended_assignments)
```

FINAL C103 RECOMMENDATION

=====

Open facilities: ['L005', 'L007', 'L013', 'L017', 'L024']

Number of facilities: 5

Daily cost: \$4,939.40

Daily emissions: 1.42 kg CO2

Demand served: 8,500 packages/day

Zones served: 30 of 30

Facility-level summary:

	assigned_facility	zones_served	packages_served	avg_distance_km	\
0	L005	6	1730	0.542333	
1	L007	6	1785	0.494833	
2	L013	6	1555	0.611333	
3	L017	6	1770	0.703833	
4	L024	6	1660	0.615333	

	avg_travel_time_mins	capacity	utilization_pct
0	1.300000	6000	28.833333
1	1.183333	4500	39.666667
2	1.483333	5500	28.272727
3	1.683333	4800	36.875000
4	1.483333	3200	51.875000

Assignment-level summary:

	zone	district	demand	assigned_facility	vehicle	\
3	Z004	East Side	280	L005	Delivery Truck - Electric	
4	Z005	East Side	295	L005	Delivery Truck - Electric	
10	Z011	South District	290	L005	Delivery Truck - Electric	
17	Z018	South District	300	L005	Delivery Truck - Electric	
20	Z021	East Side	280	L005	Delivery Truck - Electric	
26	Z027	East Side	285	L005	Delivery Truck - Electric	
0	Z001	Downtown	350	L007	Delivery Truck - Electric	
2	Z003	Downtown	310	L007	Delivery Truck - Electric	
6	Z007	West Side	260	L007	Delivery Truck - Electric	
8	Z009	West Side	240	L007	Delivery Truck - Electric	
24	Z025	Central	310	L007	Delivery Truck - Electric	
25	Z026	Central	315	L007	Delivery Truck - Electric	
12	Z013	North District	265	L013	Delivery Truck - Electric	
13	Z014	North District	250	L013	Delivery Truck - Electric	
14	Z015	North District	245	L013	Delivery Truck - Electric	
15	Z016	East Side	285	L013	Delivery Truck - Electric	
18	Z019	North District	255	L013	Delivery Truck - Electric	
29	Z030	North District	255	L013	Delivery Truck - Electric	
1	Z002	Downtown	325	L017	Delivery Truck - Electric	
9	Z010	South District	305	L017	Delivery Truck - Electric	
11	Z012	South District	285	L017	Delivery Truck - Electric	
16	Z017	West Side	265	L017	Delivery Truck - Electric	
22	Z023	South District	295	L017	Delivery Truck - Electric	
28	Z029	South District	295	L017	Delivery Truck - Electric	
5	Z006	East Side	270	L024	Delivery Truck - Electric	
7	Z008	West Side	275	L024	Delivery Truck - Electric	
19	Z020	Central	315	L024	Delivery Truck - Electric	
21	Z022	North District	260	L024	Delivery Truck - Electric	
23	Z024	West Side	270	L024	Delivery Truck - Electric	
27	Z028	West Side	270	L024	Delivery Truck - Electric	

	trips_per_day	distance_km	travel_time_mins	low_emission_zone
3	1	0.336	0.8	Yes
4	1	0.028	0.1	Yes
10	1	1.219	2.9	No
17	1	0.874	2.1	No
20	1	0.666	1.6	Yes
26	1	0.131	0.3	Yes
0	1	0.920	2.2	Yes
2	1	0.552	1.3	Yes
6	1	0.033	0.1	Yes
8	1	0.344	0.8	Yes
24	1	0.883	2.1	Yes
25	1	0.237	0.6	Yes
12	1	0.033	0.1	Yes
13	1	0.531	1.3	Yes
14	1	0.316	0.8	Yes
15	1	1.332	3.2	Yes
18	1	1.022	2.5	Yes
29	1	0.434	1.0	Yes
1	1	0.177	0.4	Yes
9	1	0.646	1.6	No
11	1	0.923	2.2	No
16	1	0.033	0.1	Yes
22	1	1.301	3.1	No
28	1	1.143	2.7	No
5	1	1.590	3.8	Yes
7	1	0.431	1.0	Yes
19	1	0.420	1.0	Yes
21	1	0.896	2.2	Yes
23	1	0.033	0.1	Yes
27	1	0.322	0.8	Yes

The final recommendation should be based on the C103 cost-minimizing MIP because it is the most directly implementable model and enforces the client-specific requirements. The recommended network opens 5 facilities: L005, L007, L013, L017, and L024. These facilities serve all 30 demand zones and all 8,500 daily packages while satisfying the 6-stop rule, electric-vehicle restrictions in low-emission zones, complete-zone service, and the 2-hour delivery-time guarantee.

This recommendation is attractive because it provides full service coverage using the minimum number of facilities needed under the 6-stop approximation. It also relies on electric vehicle assignments, which supports the sustainability goals of the project and ensures compliance in low-emission zones. The network is stable in sensitivity analysis, meaning it remains optimal under moderate fuel cost changes and service-level variations.

The recommended plan is therefore a strong baseline operating strategy for GreenCity Logistics. If leadership prioritizes cost efficiency, the cost-minimizing C103 solution should be implemented. If leadership places a stronger emphasis on emissions reduction, the goal programming or compromise Pareto solutions provide lower-emission alternatives, but those alternatives require higher operating

cost and, in some cases, more open facilities.

```
[60]: # 13.1 Executive results dashboard

fig_dash = make_subplots(
    rows=2, cols=3,
    subplot_titles=['Model Objective Comparison', 'Vehicle Fleet - CO vs_
↳Operating Cost',
                    'Facility Utilization (C103)', 'Demand by District',
                    'Route Feasibility (2-hr Window)', 'Emission Zones_
↳Distribution'],
    specs=[[{}], {}, {}], [{}], {}, {'type': 'pie'}])

# Panel 1: model objective comparison
model_names = []
model_vals = []
if 'lp_obj' in globals():
    model_names.append('LP Baseline')
    model_vals.append(lp_obj)
if 'nf' in globals() and nf.status == GRB.OPTIMAL:
    model_names.append('Network Flow')
    model_vals.append(nf.objVal)
if 'c103_cost' in globals():
    model_names.append('C103 MIP')
    model_vals.append(c103_cost['cost'])
if 'c103_goal' in globals():
    model_names.append('Goal Program')
    model_vals.append(c103_goal['cost'])

fig_dash.add_trace(go.Bar(
    x=model_names, y=model_vals,
    marker_color=[PALETTE['teal'], PALETTE['blue'], PALETTE['pink'],_
↳PALETTE['purple']][:len(model_names)],
    text=[f'${v/1e6:.2f}M' if v > 100000 else f'{v:,.0f}' for v in model_vals],
    textposition='outside'
), row=1, col=1)

# Panel 2: vehicle fleet emissions vs operating cost
veh_colors = [PALETTE['green'] if str(ft).lower() == 'electric' or 'Electric'_
↳in str(v)
               else PALETTE['orange'] if str(ft).lower() == 'hybrid' or 'Hybrid'_
↳in str(v)
               else PALETTE['red']
               for v, ft in zip(vehicle_df['vehicle_type'], vehicle_df.
↳get('fuel_type', [''] * len(vehicle_df)))]
```

```

fig_dash.add_trace(go.Scatter(
    x=vehicle_df['co2_emissions_kg_per_km'],
    y=vehicle_df['operating_cost_per_km'],
    mode='markers+text',
    marker=dict(size=np.maximum(vehicle_df['capacity_packages'] / 15, 8),
    color=veh_colors,
        line=dict(width=1, color='white')),
    text=vehicle_df['vehicle_type'].astype(str).str.replace(' - ', '<br>'),
    regex=False),
    textposition='top center', textfont=dict(size=8)
), row=1, col=2)

# Panel 3: C103 facility utilization
if 'facility_summary' in globals() and len(facility_summary) > 0:
    fig_dash.add_trace(go.Bar(
        x=facility_summary['assigned_facility'],
        y=facility_summary['utilization_pct'],
        marker_color=[PALETTE['green'] if u < 80 else PALETTE['orange'] if u <
95 else PALETTE['red']]
        for u in facility_summary['utilization_pct']],
        text=[f'{u:.0f}%' for u in facility_summary['utilization_pct']],
        textposition='outside'
    ), row=1, col=3)

# Panel 4: demand by district
dist_d = demand_df.groupby('district')['daily_demand_packages'].sum().
    reset_index()
fig_dash.add_trace(go.Bar(
    x=dist_d['district'], y=dist_d['daily_demand_packages'],
    marker_color=[PALETTE['teal'], PALETTE['blue'], PALETTE['pink'],
95 PALETTE['orange'], PALETTE['purple'], PALETTE['green']][:len(dist_d)]
), row=2, col=1)

# Panel 5: route feasibility within 120 minutes
feasible_cnt = int((distance_df1['travel_time_mins'] <=
95 MAX_DELIVERY_WINDOW_MINS).sum()) if 'distance_df1' in globals() else
95 int((distance_df['travel_time_mins'] <= MAX_DELIVERY_WINDOW_MINS).sum())
total_routes = len(distance_df1) if 'distance_df1' in globals() else
95 len(distance_df)
infeasible_cnt = total_routes - feasible_cnt
fig_dash.add_trace(go.Bar(
    x=['Feasible ( 120 min)', 'Infeasible (>120 min)'], y=[feasible_cnt,
95 infeasible_cnt],
    marker_color=[PALETTE['green'], PALETTE['red']]
), row=2, col=2)

```

```

# Panel 6: emission zones distribution
lez_count = int(emission_df['low_emission_zone'].sum())
non_lez_count = len(emission_df) - lez_count
fig_dash.add_trace(go.Pie(
    labels=['Low Emission Zone', 'Standard Zone'], values=[lez_count,
    ↪non_lez_count],
    marker=dict(colors=[PALETTE['green'], PALETTE['blue']],
    ↪line=dict(color='#0f1117', width=2)),
    textfont=dict(size=11)
), row=2, col=3)

fig_dash.update_layout(
    height=700, paper_bgcolor='#0f1117', plot_bgcolor='#1a1d2e',
    font=dict(color='#c8d0e7', size=10), showlegend=False,
    title=dict(text=' GreenCity Logistics - Executive Results Dashboard',
        font=dict(size=17, color=PALETTE['teal']), x=0.5),
    margin=dict(t=80, b=20, l=20, r=20),
)
fig_dash.show()
print('Executive dashboard rendered.')

```



Executive dashboard rendered.

The executive dashboard summarizes the most important results for decision-makers. It brings together model objectives, vehicle trade-offs, facility utilization, demand by district, route feasibility,

and emission-zone distribution. This helps translate technical optimization results into a format that managers can quickly understand.

The dashboard reinforces several important findings. First, the C103 MIP is the main recommended model because it enforces realistic facility, assignment, vehicle, and client-specific constraints. Second, electric vehicles play a central role in satisfying low-emission zone requirements. Third, the selected network serves all demand zones while maintaining delivery-time feasibility. Fourth, the trade-off analysis shows that lower-emission options are available, but they come with higher cost.

From a business perspective, this dashboard supports decision-making by connecting model outputs to operational actions: which facilities to open, which zones they serve, which vehicles to use, and what trade-offs management faces.

1.12.1 Managerial interpretation

The recommended C103 network is designed to be operationally feasible rather than only mathematically cheap. The model opens enough facilities to satisfy the 6-stop limit while maintaining complete service coverage. Low-emission zones are served only with electric vehicles, protecting the client from violating sustainability restrictions. The 2-hour delivery requirement is enforced by excluding routes that exceed 120 minutes or violate vehicle range limits.

The LP baseline provides a useful lower-cost benchmark, but it is not implementable because it allows fractional facility openings. The C103 MIP is more restrictive and therefore more realistic. The network flow validation confirms that the selected network can move the required daily package volume. Goal programming and Pareto analysis then show whether lower-emission alternatives exist and what cost premium they require.

1.12.2 Recommended decision

GreenCity Logistics should use the C103 MIP solution as the final operating plan. If leadership prioritizes sustainability more heavily than cost, the goal-programming and Pareto sections should be used to select a lower-emission alternative. If the Pareto section finds only one unique efficient solution, that means the current data and constraints do not create a meaningful cost-emission trade-off after enforcing C103 requirements.

1.13 14. Assumptions, Limitations, and Future Improvements

1.13.1 Assumptions

- 1) **One route per open facility approximation.** The C103 “maximum 6 stops per trip” rule is modeled as each open facility can serve at most 6 zones. This is a simplification because the model does not build individual multi-stop vehicle routes.
- 2) **Each zone is served completely by one facility and one vehicle type.** Demand is not split across multiple facilities. This matches C103’s “serve completely or not at all” requirement, but it simplifies real operations.
- 3) **All demand zones are served.** The model assumes GreenCity wants to serve every zone in the dataset, rather than choosing to reject low-profit or difficult zones.
- 4) **Travel times are reliable and deterministic.** Route time is treated as a fixed value. The model does not simulate traffic uncertainty, weather, accidents, or time-of-day variation beyond the available parameters.

- 5) **Two-hour delivery feasibility is based on facility-to-zone travel time.** If a route is under 120 minutes and within vehicle range, it is considered feasible.
- 6) **Electric vehicle requirement applies to low-emission zones.** Any zone marked as low-emission must use an electric vehicle.
- 7) **Distance matrix may be approximated.** If the provided distance file is incomplete, the notebook generates missing distances using latitude/longitude. These are estimated straight-line distances, not exact road-network distances.
- 8) **Vehicle capacity is handled at the daily assignment level.** The model estimates trips needed based on zone demand and vehicle capacity. It does not create detailed hour-by-hour dispatch schedules.
- 9) **Costs and emissions are based on available data.** Facility setup, transportation cost, operating cost, and CO₂ emissions are calculated from the input files. The model assumes those values are accurate.
- 10) **Facility capacity is fixed.** Each facility's package capacity is treated as known and constant.
- 11) **Demand is fixed for the planning period.** Daily demand is assumed stable during the optimization run.
- 12) **Goal programming weights reflect management priorities.** The multi-objective section assumes the chosen weights reasonably represent GreenCity's trade-off preferences.

1.13.2 Data limitations

- If the original distance matrix is incomplete, the full matrix is generated from latitude and longitude. These distances are approximations rather than actual road-network travel distances.
- The 6-stop route rule is approximated as at most 6 zones served per open facility. A richer vehicle-routing model could represent individual multi-stop routes more exactly.
- Delivery time windows are modeled using maximum travel time and feasibility filters. A full scheduling model could assign exact departure and arrival times.
- Vehicle availability is approximated using daily trip counts. A fleet-sizing extension could explicitly decide how many vehicles of each type to purchase or lease.

1.13.3 Future extensions

- Add exact vehicle-routing problem constraints for multi-stop trips.
- Add time-indexed variables for morning and afternoon delivery windows.
- Model stochastic demand and late-delivery risk.
- Use real road-network distances from a mapping API.
- Add equity or service-priority constraints for high-density neighborhoods.

1.14 15. User guide

1.14.1 How to run the notebook

1. Open this notebook in Jupyter or Google Colab.
2. Upload the five CSV files listed in Section 3.
3. Run cells from top to bottom.
4. If Gurobi asks for a license, use the academic license or the course-provided environment.

1.14.2 Main outputs

- `c103_solution_df`: zone-level recommendation showing facility, vehicle, distance, travel time, and trips.
- `facility_summary`: facility-level recommendation showing zones served, package volume, and utilization.
- `pareto_df`: cost-emissions trade-off table.
- `scenario_df`: scenario analysis table.

1.14.3 How to interpret results

- The **LP result** is a benchmark only.
- The **C103 MIP result** is the final recommended plan.
- The **network flow result** validates the selected network.
- The **goal programming and Pareto results** explain trade-offs.
- The **scenario results** show whether the recommendation is robust.